INF5130 - Algorithmique

Manuel d'exercices

Andrey Martinez Cruz

Automne 2026

Table des matières

0.1 Changement 4
0.2 Approche incrémentale 4
0.3 Annotation 4
0.4 Convention d'écriture d'algorithmes 5
0.5 Catégories d'exercices 5
0.5.1 Exercices normaux 5
0.5.2 Exercices extras 5

1 Rappel **6**
1.1 Série 1 6
1.2 Extra 9

2 Complexité asymptotique **12**
2.1 Série 2 14
2.2 Extra 15

3 Équations de récurrence **19**
3.1 Série 3 21
3.2 Extra 23

4 Diviser pour régner **25**
4.1 Série 4 26
4.2 Extra 29

5 Programmation dynamique **31**
5.1 Série 5 32
5.2 Extra 33

6 Algorithmes gloutons **35**
6.1 Série 6 36
6.2 Extra 37

7 Algorithmes sur les graphes **39**
7.1 Série 7 40
7.2 Extra 44

8 Analyse amortie **46**
8.1 Série 8 47
8.2 Extra 49

9 NP-complétude **54**
9.1 Série 9 55
9.2 Extra 55

10 Annexe **57**
10.1 Quelques propriétés logarithmiques 58
10.2 Sommation 58
10.2.1 Propriétés des sommations 58
10.2.2 Sommations particulières 59
10.2.3 Exercices 60
10.3 Démonstrations 62
10.3.1 Démonstrations générales 62
10.3.2 Démonstrations par induction 63
10.4 Corrigé 64

Préface

0.1 Changement

Au lieu de mettre individuellement les énoncés et d'attendre que le corrigé soit disponible ou d'attendre les prochaines séries, le document a été conçu pour pouvoir ajouter incrémentalement les exercices des séries à venir au cours de la session, et aussi pour donner un aperçu plus général de la matière qui sera potentiellement vue.

0.2 Approche incrémentale

Le document sera mis à jour incrémentalement. Donc, il est probable que certains exercices de certaines sections et les solutionnaires associés à ces exercices ne soient pas immédiatement disponibles, ou que certaines sections soient en cours de révision ou de rédaction.

0.3 Annotation

Afin de bien différencier les exercices qui sont plus durs que d'autres ou bien le type d'exercice correspondant, ces derniers auront un ou plusieurs des symboles suivants :

- : exercice demandant de faire des traces ou de développer les calculs ;
- : exercice portant sur des démonstrations ;
- : exercice ou chapitre en cours de révision et/ou en rédaction ;
- : exercice plus difficile que les autres.

0.4 Convention d'écriture d'algorithmes

Sauf mention contraire, tous les exercices concernant l'écriture d'algorithmes, incluant leur solutionnaire respectif, seront écrits dans une version simplifiée de **Python** avec les particularités suivantes :

- Il n'y a aucune utilisation de boucle **for**, peu importe la raison.
- Il n'y a aucune utilisation de la fonction **range**.
- Le coût de l'opération suivante :
  `t = [x] * n`

  est de $1 + \text{coût pour évaluer } x \times n$.

0.5 Catégories d'exercices

Les exercices sont divisés en deux catégories :

1. Les exercices normaux ;
2. Les exercices de la section extra.

0.5.1 Exercices normaux

Ces exercices seront ceux qui seront abordés en laboratoire.

0.5.2 Exercices extras

Ces exercices ne seront pas abordés en laboratoire (sauf par demande exceptionnelle de certaines personnes), mais seront présents pour ceux qui ont besoin de pratiquer davantage ou qui ont envie de tester davantage leurs connaissances. (Il est plus probable qu'il y ait des exercices plus durs dans cette section.) **:-)**

Chapitre 1

Rappel

1.1 Série 1

Exercice 1

On considère le problème consistant à additionner deux nombres naturels exprimés en **base deux**, dont les nombres sont représentés dans des tableaux de n éléments. Ces tableaux sont respectivement appelés A et B. La somme des deux nombres doit être calculée et placée dans un tableau nommé C et contenant $n + 1$ éléments.

a) Écrivez une fonction qui prend en entrée les tableaux A et B et renvoie le tableau C.
b) Donnez la complexité temporelle de votre algorithme.

Exercice 2

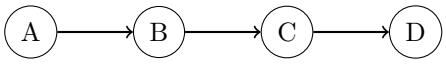
Soient A et B deux tableaux ayant la même signification que dans l'exercice précédent. On veut maintenant multiplier les nombres représentés par A et B au lieu de les additionner. Le tableau C contiendra la représentation du produit de A et B en base deux.

a) Quel doit être le nombre d'éléments du tableau C ?
b) Écrivez une fonction qui prend en entrée A et B et renvoie C.
c) Donnez la complexité temporelle de votre algorithme.

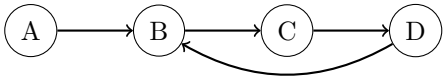
Exercice 3

Étant donnée une liste chainée L, cette dernière est cyclique si et seulement s'il existe un maillon pour lequel il existe un chemin permettant de retourner à ce dernier.

Par exemple, la liste chainée suivante ne possède pas de cycle :



En revanche, la liste chainée



en possède. Écrivez un algorithme qui renvoie **vrai** si la liste chainée L contient un cycle, sinon l'algorithme renvoie **faux**. Contrainte : votre algorithme doit être dans le pire des cas en $\mathcal{O}(n)$ pour la complexité temporelle et $\mathcal{O}(1)$ en complexité spatiale.

Pour cet exercice, vous avez accès seulement aux fonctions suivantes, qui sont exécutées en temps constant :

- OBTENIR_TETE(L) : renvoie l'emplacement du premier maillon de la liste chainée L.
- OBTENIR_SUIVANT(m) : renvoie le maillon suivant m. Si le maillon m ne contient pas de suivant, alors la fonction renvoie la valeur *None*.

Exercice 4

Soit l'algorithme de tri nommé TRI COMPTAGE, décrit par l'algorithme suivant :

```
1  def tri_comptage(A, k):
2      C = [0] * (k + 1)              # c1
3      B = [0] * (len(A))             # c2
4      j = 0                          # c3
5      while j < len(A):              # c4
6        C[A[j]] = C[A[j]] + 1        # c5
7        j = j + 1                    # c6
8      j = 1                          # c7
9      while j <= k:                  # c8
10         C[j] = C[j] + C[j - 1]     # c9
11         j = j + 1                  # c10
12     j = len(A) - 1                 # c11
13     while j >= 0:                  # c12
14       B[C[A[j]] - 1] = A[j]        # c13
15       C[A[j]] = C[A[j]] - 1        # c14
16       j = j - 1                    # c15
17     return B                       # c16
```

Ce dernier a la particularité de donner un tableau trié sans effectuer de comparaisons. Une des contraintes de cet algorithme de tri est que le tableau qui doit être trié ne contient que des nombres entiers n tels que $0 \leq n \leq k$.

a) Donnez sa complexité temporelle en fonction de $c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8, c_9, c_{10}, c_{11}, c_{12}, c_{13}, c_{14}, c_{15}$ et c_{16}.

b) Donnez sa complexité spatiale.

Exercice 5

Considérez la procédure suivante :

```python
def werk(M):
    somme = 0
    i = 0
    while i < len(M):
        stack = 0
        j = i + 1
        while j < len(M):
            stack = stack + M[i][j]
            M[i][j] = 0
            j = j + 1
        rest = stack % (i + 1)
        stack = stack // (i + 1)
        p = 0
        while p <= i:
            M[i][p] = M[i][p] + stack
            if p == i:
                M[i][p] = M[i][p] + rest
            p = p + 1
        i = i + 1
```

a) Expliquez ce que fait cet algorithme.
b) Identifiez une instruction barométrique et expliquez pourquoi il s'agit d'une opération barométrique.
c) Donnez la complexité de cet algorithme en vous basant sur l'opération barométrique trouvée à la question précédente.

Exercice 6

Soit le problème suivant pour lequel on souhaite obtenir la somme de tous les éléments présents sur la diagonale principale et la diagonale secondaire d'une matrice M carrée.

a) Écrivez un algorithme permettant de résoudre ce problème.
b) Selon l'algorithme écrit à la question précédente, identifiez une opération barométrique et expliquez ce choix.
c) Donnez la complexité de votre algorithme en vous basant sur l'opération barométrique choisie à la question précédente.

1.2 Extra

Exercice 1

Soit un tableau T composé de nombres entiers, on voudrait savoir si ce tableau possède des doublons.

a) Écrivez un algorithme qui prend en entrée un tableau d'entiers et détermine si ce tableau contient des doublons. Votre algorithme doit avoir une complexité temporelle inférieure à n^2.
b) Donnez une opération barométrique à partir de l'algorithme écrit à la question précédente.
c) Avec l'opération barométrique donnée à la question précédente, donnez la complexité de votre algorithme.

Exercice 2

Soit x, une valeur entière, et T, un tableau de nombres entiers, on voudrait savoir s'il existe deux indices $i > 0, j > 0, i, j \in \mathbb{N}, i \neq j$ tels que

$$T[i] + T[j] = x \ (i, j \in \mathbb{N})$$

a) Écrivez un algorithme qui prend en entrée un nombre entier x et un tableau d'entiers et détermine s'il existe une paire i et j qui respecte la propriété précédemment citée.
b) Donnez une opération barométrique selon l'algorithme écrit à la question précédente.
c) Donnez la complexité temporelle de votre algorithme à partir de l'opération barométrique choisie à la question précédente.

Exercice 3 

Considérez l'algorithme suivant :

```
1  def quelque_chose(n):
2      i = 0
3      while i < n:
4          j = n
5          while j > 0:
6              k = n
7              while k > 0:
8                  k = k // 2
9              j // 2
10         i = i + 1
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cet algorithme à partir de l'opération barométrique choisie.

Exercice 4

Considérez l'algorithme suivant :

```
1  def algo(t):
2      i = 0
3      m = 0
4      while i < len(t):
5          j = 0
6          element = t[i]
7          while j <= 8:
8              if j % 2 == 0:
9                  element = element + element ** 2
10             else:
11                 element = element + t[i] / j
12             j = j + 1
13         m = m + element
14         i = i + 1
15     return m
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cet algorithme à partir de l'opération barométrique choisie.

Exercice 5

Considérez l'algorithme suivant :

```
1  def bide(t):
2      i = 0
3      m = 0
4      while i < len(t):
5          i = i + 1
6          temp = [0] * len(t)
7          m = m + 1 + len(t)
8      return m
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cet algorithme à partir de l'opération barométrique choisie.

Exercice 6

Considérez l'algorithme suivant :

```
1  def autre(t):
2      i = 0
3      r = 10
4      while i < 20:
5          r = r * 2
6          i = i + 1
7      k = 0
8      while k < len(t):
9          r = r - 1
10         k = k + 1
11     return r
```

Identifiez une opération barométrique et donnez la complexité asymptotique de cet algorithme à partir de l'opération barométrique choisie.

Exercice 7

Une rotation à droite d'un tableau est un tableau dont les éléments ont été décalés de k positions vers la droite de leur position initiale. Dans le cas des éléments se trouvant à la fin du tableau, ces derniers sont remis au début du tableau pour pouvoir continuer à être décalés.

Par exemple, supposons qu'on ait le tableau $t = [2, 4, 6, 1, 2]$ et $k = 2$, alors le tableau avec rotation à droite sera $[1, 2, 2, 4, 6]$.

Écrivez un algorithme qui prend en entrée un tableau t et qui effectue une rotation vers la droite sur ce dernier et donnez la complexité temporelle et spatiale de votre algorithme.

Chapitre 2

Complexité asymptotique

Chaque algorithme peut appartenir à plusieurs classes de croissance asymptotique tant temporelle que spatiale. Ces dernières sont déterminées selon l'ordre de croissance en fonction de la variation de la taille du problème.

Soit deux fonctions $f : \mathbb{N} \to \mathbb{R}$ et $g : \mathbb{N} \to \mathbb{R}$, alors on peut avoir les propriétés suivantes :

- $f \in \mathcal{O}(g)$ si et seulement s'il existe une paire de constantes $c > 0, n_0 \in \mathbb{N}$ telles que

$$f(n) \leq cg(n) \ (\forall n, n \geq n_0)$$

- $f \in o(g)$ si et seulement s'il existe $n_0 \in \mathbb{N}$ telle que pour tout $c > 0$

$$f(n) \leq cg(n) \ (\forall n, n \geq n_0)$$

.

- $f \in \Omega(g)$ si et seulement s'il existe une paire de constantes $c > 0, n_0 \in \mathbb{N}$ telles que

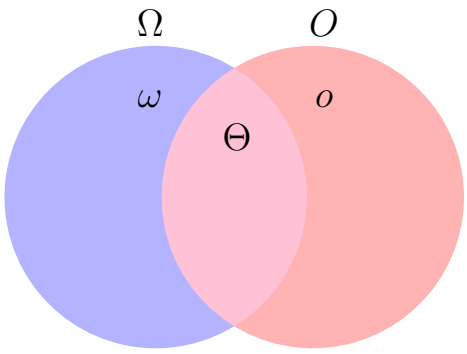
$$f(n) \geq cg(n) \ (\forall n, n \geq n_0)$$

- $f \in \omega(g)$ s'il existe $n_0 \in \mathbb{N}$ telle que pour tout $c > 0$

$$f(n) \geq cg(n) \ (\forall n, n \geq n_0)$$

- $f \in \Theta(g)$ si et seulement si $f \in \mathcal{O}(g)$ et $f \in \Omega(g)$.

Explication visuelle avec un diagramme de Venn



Ensemble de fonctions ordonnées en ordre croissant selon la vitesse de croissance :

Nom de la fonction	Fonction
Constante	$k \in \mathbb{N}$
Logarithmique	$\log n$
Polylogarithmique	$\log^k n, k > 0, k \in \mathbb{R}^+$
Linéaire	n
Quasi-linéaire	$n \log n$
Quadratique	n^2
Polynomiale	$n^k, k > 0, k \in \mathbb{N}$
Exponentielle	$k^n, k > 1, k \in \mathbb{R}^+$
Factorielle	$n!$
Hyperexponentielle	n^n

2.1 Série 2

Exercice 1

Soit les deux fonctions $f : \mathbb{N} \to \mathbb{N}$ et $g : \mathbb{N} \to \mathbb{N}$ telles que $f(n) \geq 0$ et $g(n) \geq 0$ pour tout n, prouvez que si $f \in \Theta(g)$, alors

$$f + g \in \Theta(\max(f, g))$$

en utilisant la définition de la notation Θ.

Exercice 2

Soit a et b, deux constantes réelles strictement positives, prouvez que

$$(n + a)^b \in \Theta(n^b)$$

pour n, un entier naturel, en utilisant la définition de la notation Θ.

Exercice 3

Soit c, une constante telle que $c \in \mathbb{R}$ et $c > -1$, prouvez que

$$\sum_{i=1}^{n} i^c \in \Theta(n^{c+1})$$

en utilisant la définition de la notation Θ.

Exercice 4

Classez les fonctions suivantes par ordre croissant ; autrement dit, si la fonction $f \in o(g)$, f doit être placée à gauche de g. Si $f \in \Theta(g)$, f doit être placée au-dessus ou au-dessous de g (c'est-à-dire au même niveau que g).

$$(\frac{3}{2})^n, n^3, \log^2 n, \log n!, 2^{2^n}, (\sqrt{n})^{\log(n)}, 2^{\log(n)}, (\log n)^{\log(n)}, n!, 2^{n+1}, 2^{2n}$$

Exercice 5

Pour chacun des énoncés suivants, dites s'il est **vrai** ou **faux**. Justifiez chacune de vos réponses.

a) $2n \in \Theta(n)$
b) $(\frac{n}{\log n})^2 \in o((\frac{n}{\log n})^2)$
c) $n^{\frac{1}{2}} \in \omega(\sqrt{n})$
d) $n! \in \Theta((n+1)!)$
e) $\sum_{i=1}^{n}(2i-1) \in \Omega(n^2)$
f) $2^n \in \Omega(3^n)$
g) $\sum_{i=\lfloor \frac{n}{2} \rfloor}^{n} i \in \Theta(n)$
h) $n \log n \in \Theta(n \log_4 n)$

Exercice 6

Chacun des énoncés ci-dessous est faux. Donnez un contre-exemple pour chacun d'eux.

a) Si $f \in \mathcal{O}(g)$, alors $2^f \in \mathcal{O}(2^g)$
b) $f + g \in \Theta(\min(f, g))$
c) Si $f \in \Omega(g)$, alors $g \in \Omega(f)$

2.2 Extra

Exercice 1

Démontrez que

$$\log(n!) \in \Theta(n \log n)$$

.

Exercice 2

Cet exercice consiste à comprendre la complexité asymptotique de la fonction suivante :

$$f(n) = \sum_{i=1}^{n} i^d \log^k i$$

où $d > -1, k > -1$.

a) Démontrez que

$$\sum_{i=1}^{n} \log^k i \in \Theta(n \log^k n)$$

Indice : la démonstration utilisée à l'exercice précédent est utile pour démontrer cette propriété.

b) Démontrez que

$$f \in \Theta(n^{d+1} \log^k n)$$

Indice : pour démontrer cette propriété, il est recommandé d'utiliser les démonstrations faites à l'exercice 1 et à l'exercice 3 de la série 2.

Exercice 3

Démontrez que

$$n^2 + n + \log n \notin \Omega(n^2 \log n)$$

Exercice 4

Démontrez que

$$n \in \Omega(\log^2 n)$$

Exercice 5

Démontrez que

$$n^2 + 2n \in o(n^3)$$

Exercice 6

Démontrez que

$$n^3 + n \in \Theta(\sum_{i=0}^{n} i^2)$$

Exercice 7

Soit les fonctions $f : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$ telles que

$$f(n) = \log^k \log n$$
$$g(n) = \log \log^k n$$

où k est une constante telle que $k > 0, k \in \mathbb{N}$.

Déterminez quelle est la fonction qui croît le plus vite.

Exercice 8

Démontrez que

$$n^3 + n \notin \omega(n^3)$$

Exercice 9

Démontrez que

$$3^{2n} \in \omega(3^n)$$

Exercice 10

Démontrez que

$$n! \in o(n^n)$$

sans utiliser la table de référence présente en début de chapitre.

Exercice 11

Démontrez que

$$n^2 \notin o(n \log n)$$

Exercice 12

Démontrez que

$$n \log n \notin \Theta(n \log^2 n)$$

Exercice 13

Démontrez que

$$n \log n \in \Theta(n \log_9 n)$$

Exercice 14

Soit les fonctions $h : \mathbb{N} \to \mathbb{R}^+$ et $p : \mathbb{N} \to \mathbb{R}^+$ telles que

$$h(n) = \log n!$$
$$p(n) = \log n^n$$

Déterminez si elles croissent asymptotiquement à la même vitesse.

Exercice 15

Soit trois fonctions $f_1 : \mathbb{N} \to \mathbb{R}^+, f_2 : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$, démontrez que si $f_1 \in \Theta(g)$ et que $f_2 \in \Theta(g)$, alors

$$(f_1 + f_2) \in \Theta(g)$$

Exercice 16

Démontrez que

$$\log(x^4 + x^2) \in \Theta(\log x)$$

Exercice 17

Démontrez que

$$\left\lfloor \frac{x}{2} \right\rfloor \in \omega(\log x)$$

Exercice 18

Démontrez que

$$7n^4 + 3 \in \omega(n^3)$$

Exercice 19

Le but de cet exercice est d'en apprendre un peu plus sur la complexité asymptotique des fonctions basées sur une ou plusieurs tétrations. Une tétration, notée $k \uparrow\uparrow n$ ou ${}^n k$, est un empilement de puissances de k, répété n fois. Par exemple, ${}^4 3$ ou $3 \uparrow\uparrow 4$ représentent le nombre $3^{3^{3^3}}$.

Soient deux fonctions $f : \mathbb{N} \to \mathbb{R}^+$ et $g : \mathbb{N} \to \mathbb{R}^+$ dont les définitions sont les suivantes :

- $f(n) = k \uparrow\uparrow n$
- $g(n) = k \uparrow\uparrow (n+1)$

où $k > 0, k \in \mathbb{N}$.

Est-ce que les deux fonctions croissent asymptotiquement à la même vitesse ?

Exercice 20

Démontrez que

$$n^2 \log^2(n^2 + n) \in \Theta(n^2 \log^2(n))$$

Exercice 21

Démontrez que

$$n^3 + n \in \Theta(n^3)$$

Exercice 22

Démontrez que

$$n^2 - n \in \omega(n)$$

Exercice 23

Démontrez que

$$n - 5 \in \Theta(n)$$

Exercice 24

Démontrez que

$$n^3 + 2n^2 - n \in \Theta(n^3)$$

Exercice 25

Démontrez que

$$4n^3 - 2n^2 - n \in \mathcal{O}(n^3)$$

Chapitre 3

Équations de récurrence

Les équations de récurrence sont au cœur des algorithmes récursifs.

Les deux types d'équations de récurrence qui seront étudiés dans cette section sont les équations de récurrence aux différences finies et les équations de récurrence aux divisions finies. Quelques petits cas plus exotiques seront aussi abordés.

Équations de récurrence aux différences finies

Ce type d'équation peut prendre la forme

$$\tau(n) = \alpha\tau(n - \beta) + f(n)$$

où $\alpha > 0, \beta > 0, \alpha, \beta \in \mathbb{N}$ et où $f : \mathbb{N} \to \mathbb{R}$ est la fonction de recombinaison.

Équations de récurrence aux divisions finies

Ce type d'équation peut prendre la forme

$$\tau(n) = \alpha\tau(\frac{n}{\beta}) + f(n)$$

où $\alpha > 0, \beta > 1, \alpha, \beta \in \mathbb{N}$ et où $f : \mathbb{N} \to \mathbb{R}$ est la fonction de recombinaison.

Résolution des équations de récurrence

Il existe plusieurs techniques pour évaluer une équation de récurrence.

Parmi ces techniques, en voici quelques-unes, dont certaines seront abordées en cours :

- Tenter de deviner la complexité asymptotique et prouver par induction et avec la notation Θ que l'équation de récurrence est du même ordre de grandeur que l'hypothèse de départ ou dérouler la fonction pour tenter de trouver une formule permettant d'évaluer l'équation.

- Faire un arbre des appels récursifs pour trouver une formule permettant d'évaluer la complexité asymptotique de l'équation de récurrence.
- Utiliser le théorème général, si possible, pour trouver la complexité asymptotique d'une équation de récurrence aux divisions finies.

Pour rappel, pour une fonction de récurrence aux divisions finies décrite à la page précédente, le théorème général (de base) se décline en l'un des cas suivants en comparant la complexité asymptotique de $n^{\log_\beta \alpha}$ et f :

Cas	Condition	Conclusion
1	$f \in \mathcal{O}(n^{\log_\beta(\alpha)-\epsilon})$	$\tau \in \Theta(n^{\log_\beta \alpha})$
2	$f \in \Theta(n^{\log_\beta \alpha} \log^k n), k \geq 0$	$\tau \in \Theta(n^{\log_\beta \alpha} \log^{k+1} n)$
3	$f \in \Omega(n^{\log_\beta(\alpha)+\epsilon})$ et s'il existe une paire de constantes $0 < k < 1$ et $n_0 \geq 0$ telles que $\alpha f(\frac{n}{\beta}) \leq k f(n)$ $(\forall n, n \geq n_0)$	$\tau \in \Theta(f)$

Note : $\epsilon > 0, \epsilon \in \mathbb{R}^+$.

3.1 Série 3

Exercice 1

La fonction suivante prend en entrée une portion de tableau contenue entre l'indice B_I et l'indice B_S et renvoie deux nombres entiers. Les éléments du tableau sont eux-mêmes des entiers. Vous pouvez supposer que les éléments du tableau sont tous différents les uns des autres et que le nombre d'éléments de la portion de tableau est pair et plus grand que 0.

```
1  def bidon(tab, bi, bs):
2      if tab[bi] >= tab[bi + 1]:
3          w = tab[bi]
4          x = tab[bi + 1]
5      else:
6          w = tab[bi + 1]
7          x = tab[bi]
8      if  bs <= (bi + 1):
9          n1 = w
10         n2 = x
11     else:
12         (y, z) = bidon(tab, bi + 2, bs)
13         if w >= y:
14             n1 = w
15         else:
16             n1 = y
17         if x <= z:
18             n2 = x
19         else:
20             n2 = z
21     return (n1, n2)
```

a) Supposons que $B_I = 0, B_S = 5$ et que Tab contienne les éléments {-7, 2, -3, 5, 0 , 1}. Faites la trace de l'appel BIDON(TAB, B_I, B_S) : pour chaque appel à BIDON() dans la pile d'exécution, donnez les valeurs de B_I, B_S et les valeurs retournées n_1, n_2. Si ces dernières ne sont pas encore connues, mettez « ?».

b) Décrivez ce que fait BIDON() en général.

c) Soit $T(n)$ le nombre de comparaisons effectuées par un appel de BIDON() lorsque le nombre d'éléments de la portion de tableau est égal à n. La comparaison $B_S \leq B_I + 1$ ne compte pas... Écrivez l'équation de récurrence associée à cet algorithme.

d) Résolvez l'équation de récurrence que vous avez trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercice 2

Voici un algorithme qui permet de résoudre le problème des tours de Hanoï.

```
1  def deplacer(n, t1, t2, t3):
2      if n == 1:
3          print("Déplacer un disque de la tour t1 à la tour t2")
4      else:
5          deplacer(n - 1, t1, t3, t2)
6          print("Déplacer un disque de la tour t1 à la tour t2")
7          deplacer(n - 1, t3, t2, t1)
```

DEPLACER(n, T_1, T_2, T_3) a pour effet d'afficher tous les déplacements requis pour enfiler n disques sur la tour T_2. Au départ, les disques sont enfilés sur la tour T_1.

a) Soit $T(n)$, le nombre de déplacements effectués ou, ce qui revient au même, le nombre de messages affichés par la procédure DEPLACER lorsqu'elle est appelée avec n comme paramètre. En supposant que l'affichage d'un message prend un temps égal à 1, écrivez l'équation de récurrence associée à $T(n)$.
b) Résolvez l'équation de récurrence trouvée à la question précédente. Vous pouvez la résoudre de manière exacte.

Exercice 3

Soit une matrice M composée d'entiers, un élément est considéré comme le maximum global si et seulement s'il est le plus grand élément de la matrice.

a) Écrivez un algorithme **récursif** qui prend une matrice en entrée et renvoie le maximum global de cette matrice.
b) Donnez l'équation de récurrence de votre algorithme.
c) Donnez une estimation de la complexité asymptotique de votre équation de récurrence.

Exercice 4

Écrivez un algorithme efficace pour calculer la racine carrée entière d'un entier $n \geq 2$. La racine carrée entière de n est le plus grand entier positif r tel que $r^2 \leq n$. Par «algorithme efficace», on entend un algorithme dont le temps d'exécution est proportionnel à $\log n$.

Exercice 5

Pour chacune des équations de récurrence suivantes, trouvez une fonction f aussi simple que possible telle que $\tau \in \Theta(f)$.

a) $\tau(n) = 7\tau(\frac{n}{2}) + n^3$
b) $\tau(n) = 4\tau(\frac{n}{3}) + n$
c) $\tau(n) = n\tau(n-1)$
d) $\tau(n) = 8\tau(\frac{n}{2}) + n^3$
e) $\tau(n) = 2\tau(n-1) + n^2$
f) $\tau(n) = 2\tau(\frac{n}{3}) + \log n^2$
g) $\tau(n) = 4\tau(\frac{n}{3}) + n^{\log_3 4}$
h) $\tau(n) = 7\tau(\frac{n}{3}) + n^2 \log n^3$
i) $\tau(n) = 5\tau(\frac{n}{3}) + n^{\frac{3}{2}}$

Exercice 6

Le temps d'exécution de l'algorithme A est donné par l'équation de récurrence

$$\tau_A(n) = 7\tau_A(\frac{n}{2}) + n^2$$

, alors que celui de l'algorithme A' est donné par l'équation

$$\tau_{A'} = c\tau_{A'}(\frac{n}{4}) + n^2$$

(où c est un entier positif inconnu). Supposez que $\tau_{A'} \in o(\tau_A)$. Quelle est la plus grande valeur possible pour c ?

Exercice 7

Résolvez les équations de récurrence ci-dessous **sans** utiliser le théorème général.

a) $\tau(n) = 2\tau(\frac{n}{3}) + n$
b) $\tau(n) = 4\tau(\frac{n}{2}) + \sqrt{n}$
c) $\tau(n) = 2\tau(\frac{n}{2}) + n$
d) $\tau(n) = 2\tau(\frac{n}{3}) + \log n$
e) $\tau(n) = 4\tau(\frac{n}{4}) + n \log n$

3.2 Extra

Exercice 1

Considérez l'équation de récurrence

$$\tau(n) = \tau(\sqrt{n}) + 1,$$
$$\tau(2) = 1$$

En supposant que n est de la forme 2^{2^p} pour un entier $p \in \mathbb{N}$, donnez la complexité asymptotique de cette équation.

Exercice 2

Soit t un tableau de nombres entiers distincts, trié par ordre croissant. On désire résoudre le problème suivant :

Existe-t-il un indice i tel que $t[i] = i$?

Écrivez un algorithme qui vérifie si un tel indice existe dans t. Votre algorithme doit s'exécuter dans le pire des cas en $\log n$, où n est la taille du tableau.

Exercice 3

Pour les équations de récurrence ci-dessous, donnez une fonction f telle que $\tau \in \Theta(f)$.

a) $\tau(n) = 3\tau(\frac{n}{5}) + \sqrt{n}$
b) $\tau(n) = \tau(n-8) + 2n^2$
c) $\tau(n) = 9\tau(\frac{n}{3}) + n^3$
d) $\tau(n) = \tau(\frac{n}{4}) + \log^4 n$
e) $\tau(n) = 2\tau(n-2) + 2^n$
f) $\tau(n) = 7\tau(\frac{n}{4}) + n \log n$
g) $\tau(n) = \tau(n-1) + 3^n$
h) $\tau(n) = 3\tau(\frac{n}{2}) + n^{\frac{5}{2}}$
i) $\tau(n) = 2\tau(\frac{n}{2}) + n^2$
j) $\tau(n) = 5\tau(\frac{n}{3}) + n \log^2 n$
k) $\tau(n) = 8\tau(\frac{n}{2}) + \sum_{i=0}^{n} 4^i$
l) $\tau(n) = 2\tau(\frac{n}{2}) + \sqrt{n}$
m) $\tau(n) = \tau(n-2) + n^2$
n) $\tau(n) = 8\tau(\frac{n}{4}) + n^3$
o) $\tau(n) = 9\tau(\frac{n}{3}) + n^2$
p) $\tau(n) = \tau(n-1) + \log n$
q) $\tau(n) = \tau(n-1) + n^2$
r) $\tau(n) = \tau(n-1) + n!$

Exercice 4

Le but de cet exercice est de résoudre des équations de récurrence de la forme

$$\tau(n) = \tau(n-\beta) + \tau(\beta) + n$$

où n est un nombre naturel positif.

a) Résolvez cette équation de manière exacte dans le cas où β est égal à 1. Vous pouvez supposer que $\tau(1)$ est une constante connue.
b) Résolvez cette équation de manière exacte dans le cas où β est égal à 2. Vous pouvez supposer que $\tau(2)$ et $\tau(1)$ sont des constantes connues.
c) Pouvez-vous «deviner» la complexité asymptotique de la solution lorsque β est un nombre quelconque, mais fixe ?

Chapitre 4

Diviser pour régner

Ce paradigme algorithmique se base sur le principe suivant :

- Diviser le problème en plusieurs sous-problèmes (Diviser)
- Résoudre ces sous-problèmes indépendamment (Régner)
- Combiner les résultats obtenus dans la résolution des sous-problèmes (Combiner)

Ce principe est très utile lorsque les sous-problèmes ne se chevauchent pas.

La complexité asymptotique d'un algorithme utilisant ce principe se formule sous la forme d'une équation de récurrence aux divisions finies.

4.1 Série 4

Exercice 1

Considérez le problème suivant : un tableau de taille n contient tous les nombres entiers de 0 à n **excepté un nombre**. Les éléments du tableau sont triés de façon croissante. Le problème consiste à trouver l'entier qui n'est pas dans le tableau (un nombre entre 0 et n). Voici quelques exemples :

n	tableau	entier manquant
5	$[0, 1, 3, 4, 5]$	2
5	$[1, 2, 3, 4, 5]$	0
7	$[0, 1, 2, 3, 4, 5, 6]$	7
8	$[0, 1, 2, 3, 5, 6, 7, 8]$	4

Voici un algorithme qui permet de résoudre ce problème.

```
1  def entier_manquant(tab):
2      if not tab[0] == 0:
3          return 0
4      i = 1
5      reponse = 0
6      trouve = False
7      while i < len(tab) and not trouve:
8          if tab[i] == i:
9              i = i + 1
10         else:
11             trouve = True
12     reponse = i
13     return reponse
```

a) Donnez le nombre d'exemplaires de problèmes possibles pour $n \geq 1$.
b) Existe-t-il une corrélation entre les indices du tableau et le nombre associé à la position i du tableau ?
c) Écrivez un algorithme, qui prend en entrée un tableau T et renvoie l'entier manquant, utilisant le principe «diviser pour régner» et donnez la complexité temporelle de votre algorithme avec la notation Θ.
d) Si le tableau n'était pas trié, est-ce que l'algorithme écrit à la question précédente serait encore valide ?
e) Proposez un algorithme équivalent qui prend en compte la contrainte de la question précédente.

Exercice 2

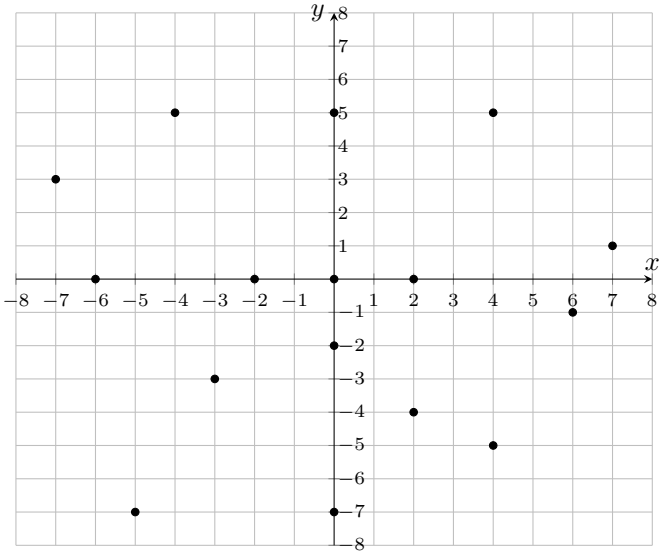
Soit T, un tableau composé d'entiers distincts triés en ordre **décroissant**, à partir duquel on voudrait créer un arbre binaire de recherche de hauteur minimale. Pour cet exercice, considérez que vous avez accès à la fonction ABR(n, G, D) qui crée un arbre dont la racine a pour valeur n, dont l'enfant gauche est l'arbre G et dont l'enfant droit est l'arbre D. Dans le cas de la création d'un nœud dont l'un des enfants est une feuille, il faut que cet enfant ait la valeur *None* en argument.

a) Écrivez un algorithme permettant de résoudre le problème en utilisant le principe «diviser pour régner».
b) Donnez l'équation de récurrence de votre algorithme.
c) À l'aide du théorème général, donnez la complexité asymptotique de votre algorithme.

Exercice 3

Soit un tableau composé de points en 2D, on souhaite obtenir la plus petite distance entre deux points faisant partie du tableau. Pour cet exercice, vous avez accès à une fonction nommée **dist**(α, β) qui calcule la distance entre les points α et β en temps constant.

a) Considérez le plan suivant, sur lequel figurent plusieurs points.



Si la plus petite distance trouvée jusqu'à présent est celle entre les points $(-6, 0)$ et $(-7, 3)$, qui est d'environ $3,16$ si on utilise la distance euclidienne, quels sont les points pour lesquels il serait plus probable de trouver une distance plus petite ?

Recommandation : il peut être utile de dessiner autour de chaque point une région pour laquelle il y aurait une possibilité de trouver une distance plus petite que la distance courante.

b) Donnez le nombre maximal de points qui pourraient potentiellement former la plus petite distance trouvée jusqu'à présent, à partir d'un certain point.

Indice : pour un point quelconque, dessinez la région dans laquelle des points pourraient se retrouver et divisez cet espace en sous-espaces plus petits en vous basant sur une distance δ, la plus petite distance trouvée

jusqu'à présent.

c) En supposant que le tableau soit trié en fonction de la coordonnée x des points, et de la coordonnée y en cas d'égalité, écrivez un algorithme utilisant le principe «diviser pour régner», qui prend en entrée un tableau de points ayant les spécificités citées précédemment et qui renvoie la plus petite distance entre deux points.

d) Si le tableau n'est pas trié, est-ce que l'algorithme écrit à la question précédente fonctionnerait ? S'il ne fonctionne pas, proposez une modification permettant à l'algorithme de s'exécuter correctement et donnez sa complexité temporelle.

Exercice 4

Considérez l'algorithme suivant représentant une variante de l'algorithme FFT :

```
1  def fft(tab):
2      if len(tab) == 1:
3          return tab
4      moitie = len(tab) // 2
5      w_n = exponentiation((-2. * pie) / (len(tab)) # ω_n^(-2iπ/n)
6      w = complex(1.0, 0.0)
7      pair = [complex(0.0, 0.0)] * moitie
8      impair = [complex(0.0, 0.0)] * moitie
9      res_pair = fft(pair)
10     res_impair = fft(impair)
11     k = 0
12     resultat = [complex(0.0, 0.0)] * len(tab)
13     while k < moitie:
14         resultat[k] = res_pair[k] + w * res_impair[k]
15         resustat[k + moitie] = res_pair[k] - w * res_impair[k]
16         w = w * w_n
17         k = k + 1
18     return resultat
```

L'algorithme prend en entrée un tableau de nombres réels et renvoie un tableau de nombres complexes. Un nombre complexe z est un nombre de la forme

$$z = a + bi \tag{4.1}$$

La constante i est l'unité imaginaire, qui a la propriété suivante : $i^2 = -1$. On note aussi que $z \in \mathbb{C}$.

La fonction EXPONENTIATION calcule l'exponentiation d'un nombre imaginaire en utilisant la formule d'Euler, dont la définition est

$$e^{ix} = \cos x + i \sin x \tag{4.2}$$

Cet algorithme a de belles propriétés et est largement utilisé, notamment dans les domaines du traitement du signal, du traitement d'image ou de l'algèbre, pour n'en citer que quelques-uns.

a) Formalisez l'équation de récurrence qui a la forme suivante :

$$\tau(n) = \alpha\tau(\frac{n}{\beta}) + f(n)$$

tout en donnant les constantes α et β telles que $\alpha > 0, \beta > 1, \alpha, \beta \in \mathbb{N}$ ainsi que la définition de la fonction f telle que $f : \mathbb{N} \to \mathbb{N}$.

b) À l'aide du théorème général, donnez une fonction $g : \mathbb{N} \to \mathbb{R}^+$ telle que $\tau \in \Theta(g)$.

4.2 Extra

Exercice 1

Considérez une matrice telle que chaque ligne est triée en ordre croissant et chaque colonne est triée en ordre croissant.

Par exemple, une instance valide de cette matrice est la suivante :

$$M = \begin{bmatrix} 1 & 4 & 7 & 8 \\ 4 & 5 & 9 & 10 \\ 8 & 10 & 14 & 15 \end{bmatrix}$$

Écrivez un algorithme permettant de vérifier si un élément x est présent dans la matrice M en utilisant le principe «diviser pour régner».

Exercice 2

Nous voulons résoudre le problème qui consiste à déterminer s'il existe une valeur x telle que

$$|\{i|T[i] = x\}| \geq \frac{n}{2}$$

où T est un tableau composé d'un type de données nommé *Objet*.

a) En supposant qu'on ait à disposition des fonctions permettant de comparer ce type de données, décrivez un algorithme dans $\mathcal{O}(n \log n)$ qui résout ce problème si on considère *Objet* comme étant le type «Entier» (donc T est un tableau d'entiers).

b) Considérons le cas général où *Objet* est un type de données quelconque et supposons que seul l'opérateur booléen d'égalité (noté $=$) soit défini pour le type *Objet*, c'est-à-dire que l'opérateur $=$ permet seulement de déterminer si deux valeurs de type *Objet* sont égales ou non (vous n'avez donc pas droit aux opérations $<, >, \leq, \geq$). Appliquez le principe «diviser pour régner» pour décrire un algorithme dans $\mathcal{O}(n \log n)$ qui résout le problème pour le type *Objet*.

Exercice 3

Une matrice booléenne M est une matrice composée uniquement de 0 et de 1.

Le produit matriciel de deux matrices booléennes A et B, noté $A \odot B$, est une matrice booléenne C dont chaque élément est

$$c_{i,j} = \bigvee_{k=1}^{m} a_{i,k} \wedge b_{k,j}$$

où $c_{i,j}$ est un élément dans la matrice résultante à la coordonnée (i, j), $a_{i,k}$ l'élément à la coordonnée (i, k) de la matrice A, $b_{k,j}$ l'élément à la coordonnée (k, j) de la matrice B et m la taille d'une ligne de la matrice A ou la taille d'une colonne dans la matrice B.

a) Donnez le produit matriciel $C = A \odot B$, si les matrices A et B sont les suivantes :

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad B = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

b) Donnez la complexité temporelle d'un algorithme de multiplication de deux matrices booléennes qui se base uniquement sur la définition mathématique.

c) Proposez un algorithme permettant de faire le produit matriciel de deux matrices booléennes en utilisant le principe «diviser pour régner».

Chapitre 5

Programmation dynamique

La programmation dynamique est une technique et un paradigme algorithmique notamment utilisés pour résoudre des problèmes d'optimisation. Si les sous-problèmes se chevauchent, la programmation dynamique devient un bon candidat pour résoudre ce type de problème.

Plutôt que de partir du problème initial et de le résoudre en divisant le problème en sous-problèmes, la programmation dynamique part du cas de base et résout les sous-problèmes de manière ascendante.

L'approche utilisée par ce paradigme algorithmique est dite *bottom-up*, contrairement à des approches comme «diviser pour régner», qui sont dites *top-down*.

5.1 Série 5

Exercice 1

Trouvez la plus longue sous-suite commune entre les mots **algorithmie** et **isomorphisme**.

Exercice 2

Trouvez la distance de Levenshtein et un alignement pour les mots **scinder** et **cisailler**. Supposez que le coût de l'insertion, de la suppression et du changement de caractère est de 1.

Exercice 3

Trouvez la distance de Levenshtein pour les séquences **CGATAGAT** et **GTCCAG** si les coûts des différentes opérations sont les suivants :

- Insertion : 3
- Suppression : 2
- Changement de caractère :

	A	G	T	C
A	0	2	1	3
G	2	0	2	1
T	1	2	0	2
C	3	1	2	0

Exercice 4

Les nombres de Catalan sont définis ainsi :

$$C_n = \begin{cases} 1 & \text{si } n = 0 \\ \sum_{i=1}^{n} C_{i-1} C_{n-i} & \text{sinon} \end{cases} \tag{5.1}$$

Écrivez un algorithme de programmation dynamique permettant de calculer le n-ième nombre de Catalan.

Exercice 5

Soit le problème qui consiste à chercher le plus grand carré de 1 dans une matrice carrée A dont les coefficients valent 0 ou 1.

Par exemple, dans la matrice suivante :

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

l'algorithme devrait renvoyer 3.

Donnez un algorithme de programmation dynamique qui prend en entrée une matrice A respectant les contraintes du problème et renvoie la largeur maximale k d'un carré de 1 dans A.

5.2 Extra

Exercice 1

Trouvez la plus longue sous-suite commune entre les mots **constitutionnellement** et **constipation**.

Exercice 2

Trouvez la plus longue sous-suite commune entre les mots **touchable** et **disponible**.

Exercice 3

Trouvez la distance de Levenshtein et un alignement pour les mots **titubiler** et **tisane**.

Exercice 4

Trouvez la distance de Levenshtein et un alignement pour les mots **actuellement** et **couramment**.

Exercice 5

Un voyageur souhaite se rendre à la ville v_n à partir de la ville v_1. Pour cela, il a décidé de prendre sa voiture, qui utilise de l'essence. Sachant que la distance pour se rendre à une ville v_p à partir d'une ville v_w n'est pas nécessairement équivalente à la consommation d'essence pour s'y rendre, et qu'il ne veut évidemment pas retourner à une ville précédente, il aimerait savoir quel serait l'itinéraire le plus court qui limiterait sa consommation d'essence. La matrice des distances D et la matrice de consommation d'essence C sont représentées ainsi :

$$D, C = \begin{bmatrix} 0 & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ \infty & 0 & a_{2,3} & \cdots & a_{2,n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \infty & \infty & \infty & \cdots & 0 \end{bmatrix}$$

a) Donnez le schéma dynamique de ce problème.

   Indice : $\min_{i=m}^{n} a_i$ cherche le minimum parmi les éléments a_m à a_n.

b) En vous basant sur le schéma dynamique trouvé à la question précédente, écrivez un algorithme utilisant la **programmation dynamique** qui prend en entrée les éléments suivants :

   1. La matrice des distances entre les villes D, dont toutes les cases sont initialisées à ∞ ;
   2. La matrice de consommation d'essence C.

Exercice 6

Considérez le problème suivant : dans un bâtiment de n étages, avec k œufs en notre possession, on souhaite connaître l'étage le plus élevé de ce bâtiment duquel un œuf peut être lancé sans se briser à l'impact. Cet étage se nomme l'étage critique.

Les règles concernant les œufs sont les suivantes :

- Si l'œuf se casse, alors tout autre lancer d'œuf à un étage supérieur à cet étage brisera aussi les œufs restants.
- Si l'œuf ne se casse pas, alors aucun lancer à un étage inférieur à celui-ci ne brisera d'œuf. L'œuf lancé peut être réutilisé dans ce cas-ci.

Pour les trois premières questions, on supposera que $k = 2$.

a) Si on suppose que le nombre d'étages est de 36, donnez l'étage critique.
b) Donnez le schéma de programmation dynamique.
c) Écrivez un algorithme de programmation dynamique qui se base sur le schéma dynamique donné à la question précédente.
d) Si on suppose qu'on a k œufs à disposition, donnez une généralisation de cette formule.

Chapitre 6

Algorithmes gloutons

Les critères d'un algorithme glouton sont les suivants :

- L'algorithme est «myope». Il ne considère que le meilleur résultat localement.
- L'algorithme pioche dans un ensemble de candidats possiblement meilleurs que la solution courante et, si l'un d'eux est pire que la meilleure solution courante, alors il ne sera plus considéré tout au long de l'algorithme.

Un algorithme glouton n'est pas nécessairement optimal, mais il peut donner une solution proche de la ou des solutions optimales.

6.1 Série 6

Exercice 1

Soit le tableau suivant, qui contient des lettres et leurs fréquences :

Lettre	a	b	c	d	e	f	g	h	i
Fréquence	26	24	13	12	7	6	5	4	3

a) Construisez un code de longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercice 2

Dans quel ordre faut-il effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	3	2	2	3	4	5	1	7
Pénalité	95	85	55	60	75	45	40	30

Exercice 3

Nous considérons une variante du problème de la petite monnaie. Il s'agit de déterminer s'il existe des entiers naturels x_i tels que

$$\sum_{i=1}^{n} p_i x_i = P$$

où les p_i, les x_i et P ont la même signification que dans les diapositives du cours. Nous supposons que les p_i sont en ordre croissant. Notez que si p_1 n'est pas égal à 1, l'équation ci-dessus peut ne pas avoir de solution. En d'autres termes, il peut s'avérer impossible de rendre la monnaie, même si on dispose de quantités illimitées de chaque type de pièce. Pour rendre la monnaie, on applique l'algorithme glouton qui consiste à choisir d'abord les pièces de plus grande valeur. S'il n'est pas possible de rendre la monnaie qui reste après avoir considéré $p_n, p_{n-1}, \cdots, p_2, p_1$, l'algorithme glouton conclut qu'on ne peut pas rendre la monnaie.

a) L'algorithme glouton donne-t-il la bonne réponse si $n = 3, p_1 = 2, p_2 = 5, p_3 = 10$ et $P = 6$?

b) L'algorithme glouton donne-t-il la bonne réponse si $n = 3, p_1 = 2, p_2 = 6, p_3 = 8$ et $P = 12$?

c) Supposons maintenant que, comme dans la «vraie vie», le nombre de pièces de chaque type soit limité. Écrivez l'algorithme qui prend en compte ces contraintes. Supposez que l'algorithme prend en argument des couples (p_i, b_i) où p_i représente la valeur de la pièce p_i et b_i la quantité disponible de pièces p_i.

Exercice 4

Soit le problème du palindrome suivant. Étant donné une liste de caractères en minuscules L, quel est le plus long palindrome qui peut être formé avec les caractères de la liste L ?

Par exemple, si $L =$ **['a', 'b', 'c', 'c', 'c', 'c', 'd', 'd']**, alors un des plus longs palindromes que l'on peut construire est *dccaccd*, dont la longueur est de 7.

Écrivez un algorithme glouton qui prend en paramètre une liste de caractères L et qui retourne un entier représentant la taille du plus long palindrome qu'on peut construire avec les caractères de la liste L.

6.2 Extra

Exercice 1

Soit le tableau suivant, qui contient des lettres et leurs fréquences :

Lettre	a	b	c	d	e	f	g	h
Fréquence	12	16	13	12	18	17	15	15

a) Construisez un code de longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercice 2

Soit le tableau suivant, qui contient des lettres et leurs fréquences :

Lettre	a	b	c	d	e	f	g	h	i
Fréquence	20	8	9	10	12	11	14	9	7

a) Construisez un code de longueur minimale fixe pour ces lettres et donnez sa longueur.

b) Construisez le code de Huffman pour ces lettres et donnez sa longueur.

Exercice 3

Dans quel ordre faut-il effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	3	2	2	3	4	5	1	7
Pénalité	55	45	65	80	85	95	60	70

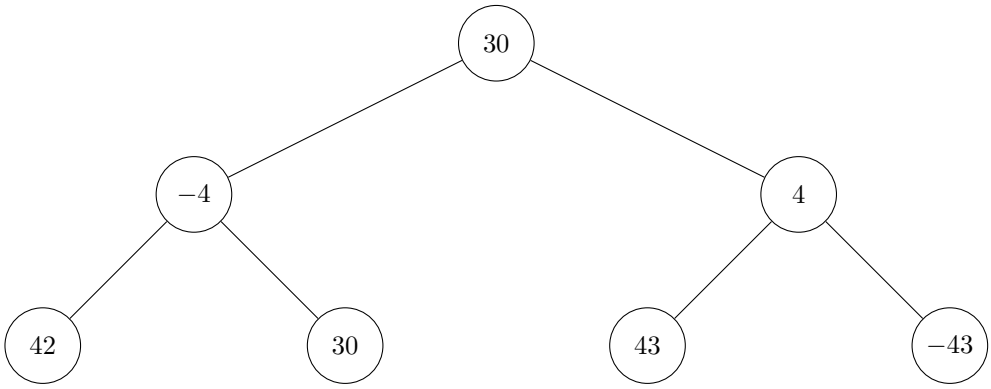
Exercice 4

Dans quel ordre faut-il effectuer les tâches pour minimiser la somme des pénalités des tâches en retard ? On suppose que les tâches ont une durée de 1 et que les échéances et les pénalités sont données dans le tableau suivant :

Tâche	1	2	3	4	5	6	7	8
Échéance	4	2	5	8	4	6	1	7
Pénalité	45	55	70	10	20	35	45	65

Exercice 5

Considérez le problème suivant : à partir d'un arbre binaire complet dont chaque nœud possède un entier $n \in \mathbb{Z}$, on souhaite trouver le chemin allant de la racine à une feuille dont la somme des valeurs est maximale. Par exemple, pour l'arbre suivant :



La somme maximale est de 77. En effet, le chemin à prendre, en partant de la racine, donne $30 + 4 + 43 = 77$, soit en allant à droite, puis à gauche.

Pour les besoins de l'algorithme, vous avez accès aux fonctions suivantes s'exécutant en temps constant :

- OBTENIR_GAUCHE(N) : renvoie l'enfant de gauche du nœud N. La fonction renvoie *None* si N est une feuille.
- OBTENIR_DROITE(N) : renvoie l'enfant de droite du nœud N. La fonction renvoie *None* si N est une feuille.
- OBTENIR_CONTENU(N) : donne le contenu du nœud N.

a) Écrivez un algorithme glouton permettant de résoudre le problème.
b) Si votre algorithme est optimal, prouvez-le. Dans le cas contraire, donnez un cas où l'algorithme ne donne pas la solution optimale.

Chapitre 7

Algorithmes sur les graphes

Rappel : un graphe noté G est un couple d'ensembles S et A définis ainsi :

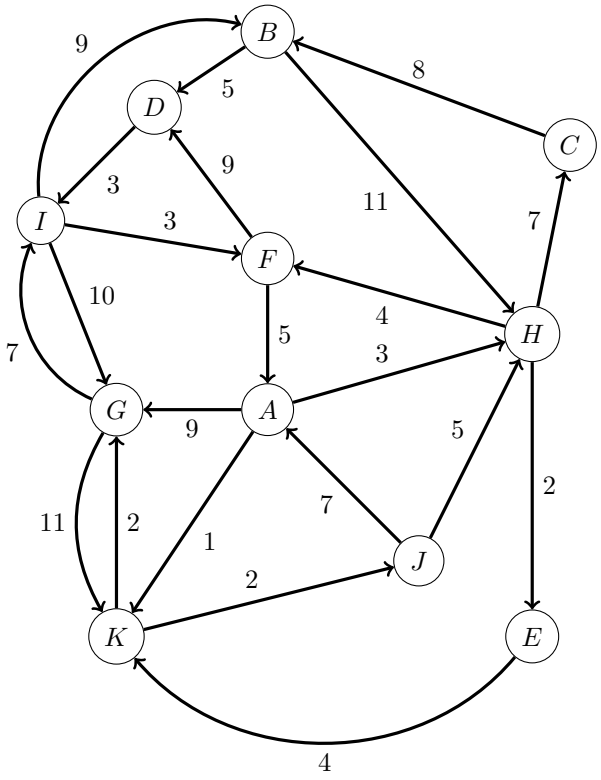
- S : l'ensemble des sommets que le graphe G contient.
- A : l'ensemble des arêtes que le graphe G contient. Une arête relie deux sommets entre eux. Une arête peut aussi relier un sommet à lui-même.

Un algorithme sur un graphe dépend souvent de deux ensembles dont les tailles peuvent être d'ordres de grandeur différents. Ainsi, à moins qu'un algorithme ne dépende que d'un des deux ensembles, la complexité d'un algorithme sur les graphes dépend de $|S|$ et de $|A|$.

7.1 Série 7

Exercice 1

Considérez le graphe suivant :



a) Effectuez une recherche en profondeur si le sommet de départ est A
b) Effectuez une recherche en largeur si le sommet de départ est A
c) Effectuez l'algorithme de Prim si le sommet de départ est A
d) Effectuez l'algorithme de Kruskal

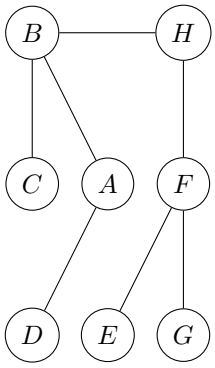
Exercice 2

Un arbre, noté T, est un graphe ayant les propriétés suivantes :

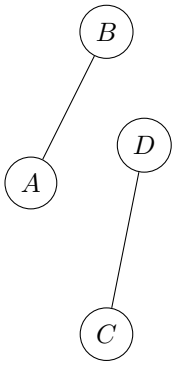
1. $|A| = |S| - 1$
2. Le graphe est acyclique, c'est-à-dire qu'il est dépourvu de cycle

a) Déterminez si les graphes suivants sont des arbres :

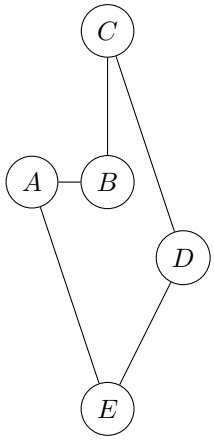
Graphe 1



Graphe 2



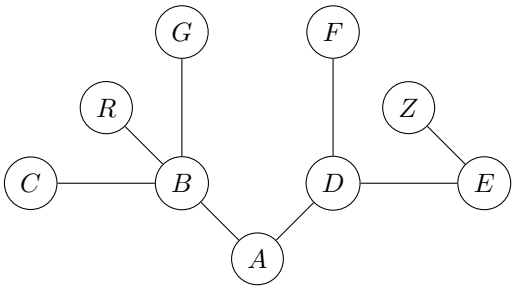
Graphe 3



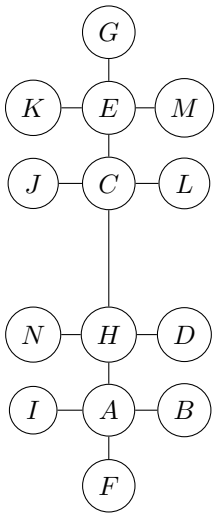
b) Un moyen de déterminer si un graphe est un arbre, et de trouver son ou

ses centres, consiste à utiliser la définition $|A| = |S| - 1$. Pour ce faire, on vérifie d'abord que le nombre d'arêtes est égal au nombre de sommets moins 1. Ensuite, à chaque tour, on «épluche» tous les sommets de degré 1 et on ajuste le degré de leurs voisins en réduisant ce degré par le nombre de sommets «épluchés» qui leur sont adjacents. On répète ce processus jusqu'à ce qu'il reste seulement 1 ou 2 sommets, ou jusqu'à ce qu'il n'y ait plus aucun sommet de degré 1. Dans ce dernier cas, le graphe n'est pas un arbre.

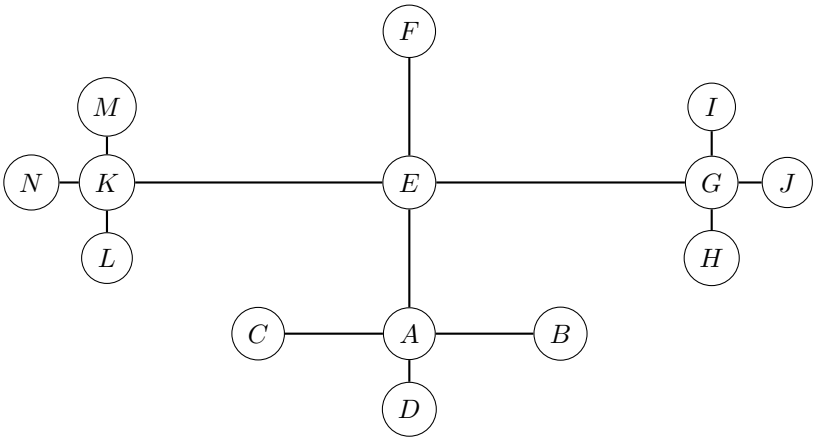
Graphe 1



Graphe 2



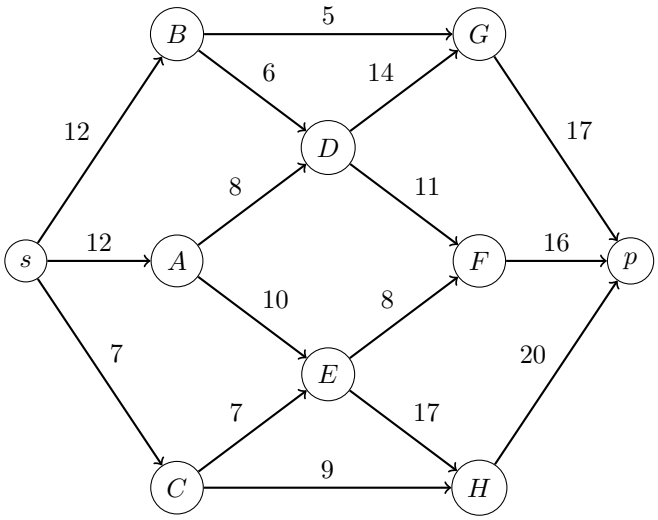
Graphe 3



c) Écrivez un algorithme qui prend en entrée un graphe G et vérifie si ce graphe est un arbre.

Exercice 3

Considérez le graphe suivant :

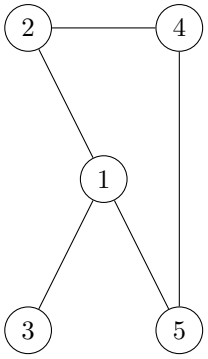


a) Trouvez le flot maximal en utilisant la variante d'**Edmonds-Karp**.
b) Trouvez le flot maximal en utilisant **une recherche en profondeur**.

7.2 Extra

Exercice 1

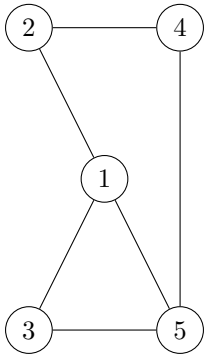
Une articulation est un sommet d'un graphe non orienté dont le retrait entraîne l'apparition de plusieurs sous-graphes non connectés entre eux. En d'autres termes, le graphe devient non connexe. Par exemple, le graphe suivant :



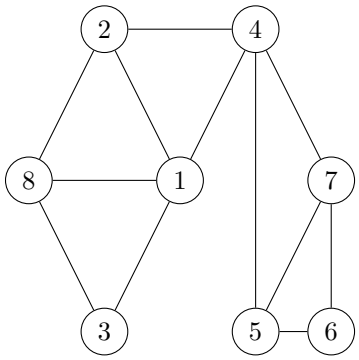
possède un point d'articulation qui est le sommet 1.

a) Déterminez si les graphes suivants possèdent une articulation.

1.



2.



b) Proposez un algorithme qui, étant donné un graphe non orienté, vérifie si ce dernier possède un point d'articulation.

Indice : une approche possible est de construire dynamiquement un graphe en forme d'arbre et de vérifier s'il existe des arêtes qui sont des *back-edges*. Ce type d'arête est une arête reliant un sommet u à un sommet v telle que u est un successeur de v dans l'arbre, mais un prédécesseur de v dans le graphe d'origine.

Chapitre 8

Analyse amortie

- Analyse classique : la complexité asymptotique est déterminée par l'instruction la plus coûteuse sur une opération quelconque.
- Analyse amortie : la complexité amortie est déterminée par la moyenne des coûts sur une séquence de n opérations.

Voici les méthodes possibles pour effectuer une analyse amortie :

- Méthode de l'agrégat : déterminer le coût amorti moyen d'une séquence de n opérations. Le coût amorti est donné par $\tau(n)/n$ où $\tau(n)$ est la somme des coûts de chaque opération dans la séquence.
- Méthode du comptable : supposer que le coût amorti est égal à une constante y et vérifier si la différence entre le coût amorti cumulé et le coût réel est ≥ 0. Analogie : faire des économies et dépenser ces dernières au moment où il faut payer cher.
- Méthode du potentiel : soit $\Phi(D_i)$, une fonction de potentiel permettant de représenter l'état de la structure de données après i opérations, le coût amorti est donné par

$$\sum_{i=1}^{n} ca(i) = \sum_{i=1}^{n} (cr(i) + \Phi(D_i) - \Phi(D_{i-1}))$$

.

Note : il n'y a pas de technique absolue pour trouver une fonction de potentiel et elle n'est pas unique, mais il est possible d'en trouver une selon le contexte du problème.

8.1 Série 8

Exercice 1

Considérez une structure de données dont les coûts des opérations sont les suivants :

$$cr(i) = \begin{cases} i+1 & \text{si } i \text{ est une puissance de 4} \\ 1 & \text{sinon} \end{cases}$$

a) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode de l'agrégat
b) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode du comptable
c) Effectuez l'analyse amortie de cette structure de données en utilisant la méthode du potentiel

Exercice 2

Une manière de représenter un «dictionnaire» est la suivante : on a un ensemble de tableaux dont la taille est une puissance de 2. Ces derniers sont soit vides, soit pleins. Dans le cas où ils sont pleins, ils sont triés. En revanche, bien qu'il y ait plusieurs tableaux triés, ceux-ci n'ont aucune relation entre eux.

Voici un exemple de «dictionnaire» à 11 éléments :

R_1	$[6]$
R_2	$[2, 10]$
R_3	Vide
R_4	$[-1, 2, 5, 7, 8, 9, 10, 25]$

Pour savoir quels sont les tableaux qui sont pleins, il faut regarder la représentation binaire du nombre d'éléments présents. Ici,

$$(11)_{10} = (1011)_2,$$

et les puissances de 2 qui sont marquées à 1 dans la représentation binaire du nombre d'éléments indiquent les tableaux pleins. Ici, les puissances 0, 1 et 3 signifient qu'il y a un tableau plein de chacune des tailles suivantes : 1, 2 et 8. Lors de l'insertion d'un nouvel élément, les deux situations suivantes peuvent se produire :

1. S'il n'y a pas de tableau de taille 1 déjà présent, alors on crée simplement un tableau de taille 1.

2. Dans le cas où un tableau de taille 1 est déjà présent, alors on fusionne ces deux tableaux en mettant les éléments dans l'ordre croissant. Dans le cas de la fusion de deux tableaux triés (de même taille dans ce cas-ci), il est possible de les fusionner en $\Theta(p)$, où p est le nombre d'éléments des deux tableaux. Ensuite, puisque le nouveau tableau a doublé de taille, il faut vérifier s'il existe un tableau de même taille que le nouveau tableau. Si c'est le cas, on répète le même processus jusqu'à ce que le tableau créé, de taille 2^c, où c est une constante, soit le seul tableau de cette taille.

Par exemple, si on ajoute l'élément 3 dans la structure précédente, on obtient le résultat suivant :

R_1	Vide
R_2	Vide
R_3	$[2, 3, 6, 10]$
R_4	$[-1, 2, 5, 7, 8, 9, 10, 25]$

Or, $(12)_{10} = (1100)_2$, ce qui signifie qu'il y a seulement un tableau plein pour chacune des tailles suivantes : 4 et 8. Pour rechercher un élément, il est possible d'effectuer une recherche dichotomique dans chaque tableau, jusqu'à ce que l'élément soit trouvé ou que tous les tableaux aient été parcourus.

a) Donnez la complexité dans le pire des cas pour l'insertion d'un nouvel élément, ainsi que sa complexité amortie.
b) Donnez la complexité de la recherche d'un élément dans cette structure de données.

8.2 Extra

Exercice 1

Supposons que le coût d'une opération sur une certaine structure de données soit le suivant :

$$cr(i) = \begin{cases} \frac{i}{8} + 1 & \text{si } i \mod 16 = 0 \\ 1 & \text{sinon} \end{cases} \tag{8.1}$$

Donnez la complexité amortie de cette structure de données.

Exercice 2

En classe, vous avez certainement vu le monceau binomial, qui a les complexités amorties suivantes, en fonction de n, le nombre d'éléments dans le monceau :

Opération	Complexité amortie
Insertion d'un élément	$\Theta(1)$
Extraction du minimum/maximum	$\Theta(\log n)$
Fusion de deux files binomiales	$\Theta(\log n)$

Or, il est possible d'améliorer l'insertion d'un élément et la fusion de deux files binomiales en garantissant qu'elles s'exécutent en temps constant tout en gardant la complexité amortie de l'extraction du minimum/maximum que le monceau binomial offre. Pour cela, les opérations se déroulent ainsi :

- Insertion d'un élément : insérer l'élément dans la file sans effectuer aucune fusion et mettre à jour le minimum ou le maximum si nécessaire.
- Fusion de deux files binomiales : mettre tous les arbres d'une file dans l'autre file et mettre à jour le minimum ou le maximum si nécessaire sans fusionner les arbres entre eux. Note : avec des listes chainées, il est possible de les fusionner en temps constant.
- Extraction du minimum/maximum : même approche que dans le cas d'un monceau binomial régulier.

On nommera cette variante du monceau binomial un monceau binomial paresseux. Pour les exercices suivants, on supposera que le monceau est un monceau min.

a) Afin de vous familiariser avec cette variante, effectuez les opérations suivantes (dans l'ordre) et donnez à chaque étape l'état de la file :

1. Insertion de 7
2. Insertion de 3
3. Insertion de 4
4. Insertion de 10
5. Insertion de 5
6. Suppression du minimum
7. Insertion de 40

8. Insertion de 57
9. Suppression du minimum
10. Insertion de 23
11. Suppression du minimum

b) Effectuez une analyse amortie de cette structure de données.

Exercice 3

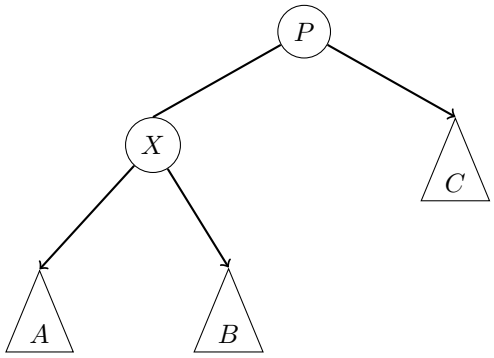
Dans le cours INF3105, vous avez certainement vu les arbres de recherche qui s'équilibrent eux-mêmes (arbres AVL, rouge-noir et arbres-B). Ces arbres garantissent une recherche, une insertion et une suppression en temps logarithmique. Cependant, ces arbres souffrent d'un défaut particulier : il est possible que les éléments les plus recherchés se trouvent en bas de l'arbre, ce qui peut devenir coûteux. Une structure de données qui permet de placer les éléments les plus recherchés le plus près possible de la racine est un arbre évasé (*splay tree*).

Un arbre évasé a la particularité d'utiliser une opération spéciale nommée *évasement* (*splay*). Cette opération peut se décomposer en une des trois sous-opérations suivantes :

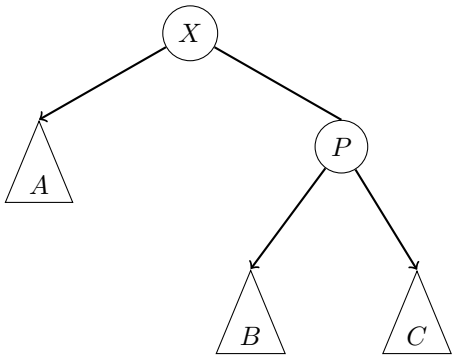
- *Zig*
- *Zig-zag*
- *Zig-zig*

Important : les opérations sont symétriques, c'est-à-dire qu'on obtient leurs variantes en les effectuant dans l'autre sens.

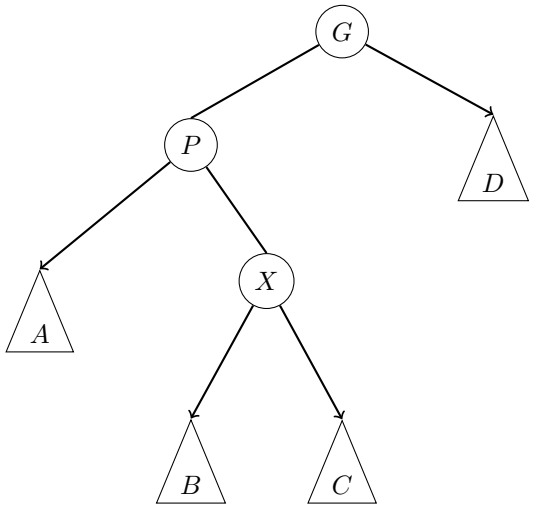
L'opération *zig* consiste à faire une rotation simple pour les nœuds X et P si P est le parent de X et que P est la racine de l'arbre. Par exemple, l'arbre



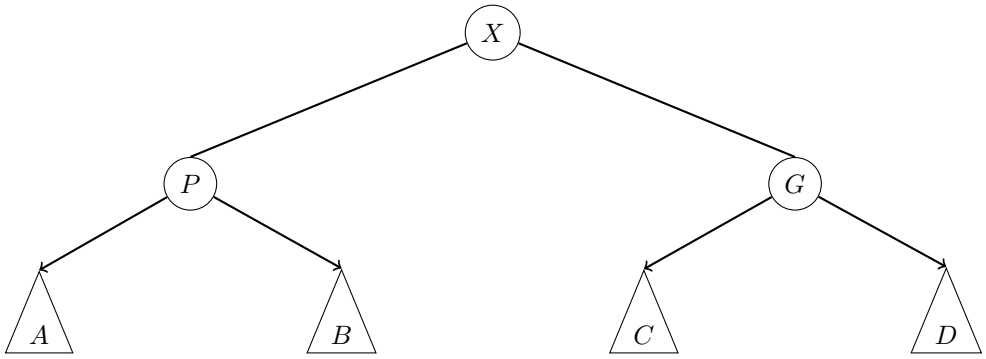
devient, après une opération *zig*, l'arbre suivant :



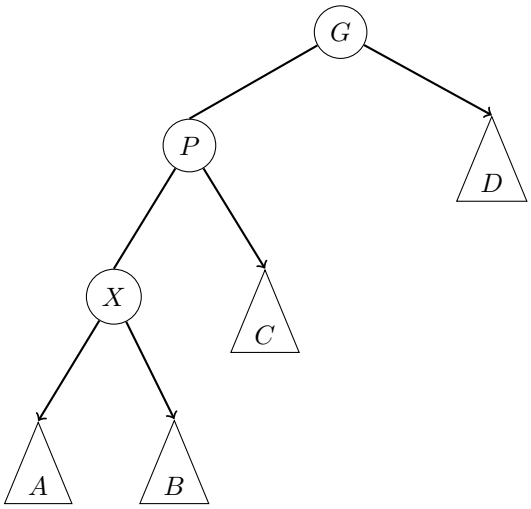
L'opération *zig-zag* consiste à faire une double rotation (droite-gauche ou gauche-droite) comme s'il s'agissait d'un arbre AVL. Par exemple, l'arbre



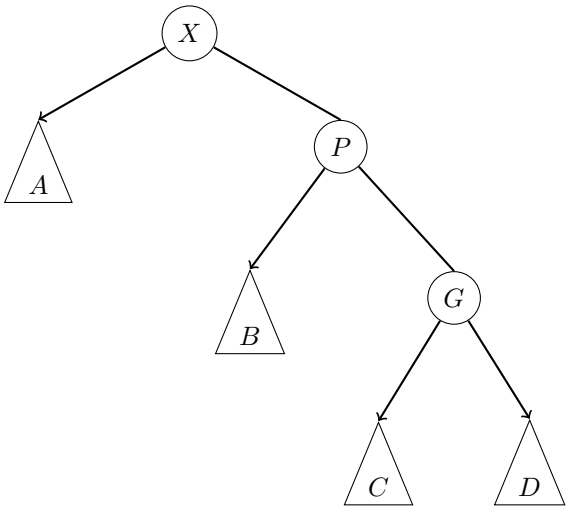
devient l'arbre suivant après cette opération :



L'opération *zig-zig* consiste à faire une double rotation du type droite-droite ou gauche-gauche. Par exemple, l'arbre



devient l'arbre suivant après l'opération :



Les opérations de base se font de la sorte :

- Insertion d'un nouvel élément : il faut insérer l'élément comme dans un arbre binaire de recherche classique, puis effectuer les opérations décrites ci-dessus jusqu'à ce que le nouvel élément soit la racine de l'arbre.
- Suppression d'un élément : on doit effectuer une suite d'opérations d'évasement jusqu'à ce que l'élément à supprimer soit la racine de l'arbre, puis on le supprime. Ensuite, il faut évaser soit le plus grand élément qui est inférieur à l'élément supprimé, soit le plus petit élément qui est supérieur à l'élément supprimé.
- Recherche d'un élément dans l'arbre : la procédure reste la même, sauf que, si l'élément recherché existe, il faut évaser l'élément trouvé. Dans le

cas où l'élément n'existe pas, il faut évaser le dernier élément qui a été comparé.

Pour les exercices suivants, on considérera la première possibilité dans le cas de la suppression d'un élément.

a) Afin de vous familiariser avec les opérations de cette structure de données, effectuez les opérations suivantes (dans l'ordre) et donnez la forme de l'arbre :
   1. Insertion de 10
   2. Insertion de 40
   3. Insertion de 30
   4. Insertion de -4
   5. Recherche de 30
   6. Recherche de -10
   7. Insertion de 50
   8. Insertion de -32
   9. Suppression de 10
   10. Suppression de 30

b) Effectuez l'analyse amortie des opérations d'évasement.

c) En vous basant sur l'analyse amortie des opérations d'évasement, donnez la complexité amortie des opérations de recherche d'un élément dans l'arbre, d'insertion d'un nouvel élément et de suppression d'un élément.

Indice :

- Une idée de fonction de potentiel serait de représenter l'état de cette structure de données de sorte que la fonction donne une grande valeur lorsqu'un élément est proche de la racine et une très faible valeur lorsqu'il en est très éloigné.
- Utiliser un logarithme pour représenter cette fonction de potentiel est très utile.

Chapitre 9

NP-complétude 

Problème P : Problème pour lequel il existe un algorithme en temps polynomial permettant de le résoudre.

Problème NP : Problème pour lequel il est facile de vérifier si une solution x résout le problème.

Un des 7 problèmes du millénaire : $P \stackrel{?}{=} NP$.

NPC : problèmes les plus durs de NP.

9.1 Série 9

Exercice 1

Soit l'instance SAT suivante :

$$\phi = ((\neg x_1 \wedge x_2) \rightarrow (x_2 \vee (x_1 \rightarrow x_5))) \rightarrow (x_4 \vee (\neg x_3 \wedge x_5))$$

a) Réduisez cette instance SAT en une instance 3-FNC-SAT.
b) Avec l'instance 3-FNC-SAT trouvée à la question précédente, convertissez cette instance en une instance du problème *CLIQUE*.
c) Trouvez au moins **deux** cliques qui satisfont ce problème.

Exercice 2

Pour les instances suivantes du problème 2-FNC-SAT, déterminez si les formules sont satisfaisables.

a) $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (x_4 \vee \neg x_3) \wedge (x_1 \vee x_4) \wedge (x_5 \vee \neg x_4) \wedge (x_2 \vee \neg x_5)$
b) $(x_1 \vee x_2) \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (x_4 \vee \neg x_5) \wedge (x_5 \vee x_6) \wedge (\neg x_5 \vee x_1) \wedge (x_5 \vee \neg x_6)$

Exercice 3

Démontrez que la borne inférieure d'un algorithme de tri se basant sur la **comparaison** est $\Omega(n \log n)$ et qu'elle est sérrée.

9.2 Extra

Exercice 1

Soit l'instance SAT

$$\phi = ((x_1 \vee \neg x_2) \wedge ((x_1 \vee x_3) \vee (\neg x_2 \wedge \neg x_3)))$$

Réduisez cette instance SAT en une instance 3-FNC-SAT.

Exercice 2

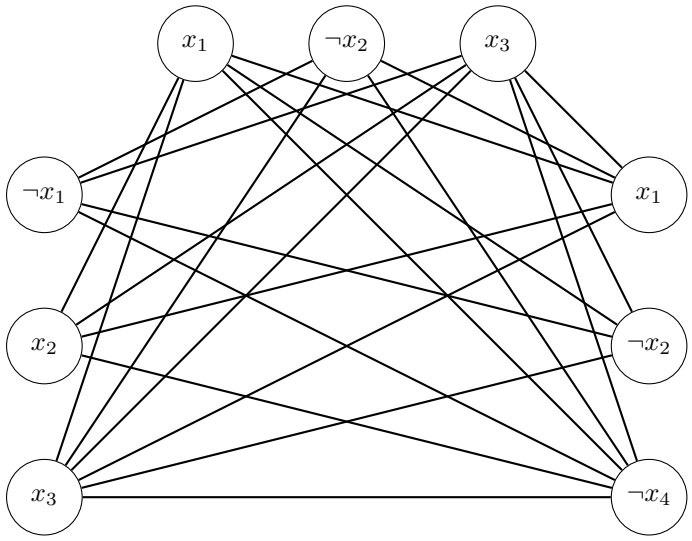
Soit l'instance SAT

$$\phi = ((x_1 \rightarrow x_3) \vee (x_2 \rightarrow x_4)) \wedge (\neg x_1 \vee (\neg x_2 \rightarrow x_3))$$

Réduisez cette instance SAT en une instance 3-FNC-SAT.

Exercice 3

Trouvez une clique valide dans cette instance du problème *CLIQUE*.



Chapitre 10

Annexe

Cette partie comporte tous les rappels mathématiques nécessaires pour effectuer les exercices des sections précédentes. Il n'est pas obligatoire de faire ces exercices, mais il est **fortement** recommandé de les faire si ces rappels ne vous disent rien ou si les notions ne sont plus qu'un lointain souvenir. C'est aussi ici que se trouve le corrigé des différents exercices de ce manuel.

10.1 Quelques propriétés logarithmiques

Soit $b \in \mathbb{R}^+ \backslash \{0, 1\}$ et $x > 0, y > 0$, voici une liste de propriétés des logarithmes :

- Valeur du logarithme de 1 :
$$\log_b(1) = 0$$
- Logarithme naturel :
$$\ln(x) = \log_e(x)$$
- Logarithme avec même base et même coefficient :
$$\log_b(b^x) = x$$
- Produit dans un logarithme :
$$\log_b(xy) = \log_b(x) + \log_b(y)$$
- Quotient dans un logarithme :
$$\log_b(\frac{x}{y}) = \log_b(x) - \log_b(y)$$
- Puissance dans un logarithme :
$$\log_b(x^y) = y \log_b(x)$$
- Changement de base :
$$\log_b(x) = \frac{\log_k x}{\log_k b}$$
- Réduction des logs en puissance :
$$b^{\log_b(x)} = x$$
- Rotation des logs en puissance :
$$a^{\log_b(x)} = x^{\log_b(a)}$$
- Passage en forme de puissance :
$$\log_b x = y \Leftrightarrow x = b^y$$
Cette règle s'applique aussi dans l'autre sens.

10.2 Sommation

10.2.1 Propriétés des sommations

Soit $f : \mathbb{N} \to \mathbb{R}$

Sommation avec une constante

$$\sum_{i=s}^{n} c \times f(i) = c \sum_{i=s}^{n} f(i)$$

Commutativité et associativité des sommations

Soit $g : \mathbb{N} \to \mathbb{R}$

$$\sum_{i=s}^{n} (f(i) \pm g(i)) = \sum_{i=s}^{n} f(i) \pm \sum_{i=s}^{n} g(i)$$

Fragmentation d'une sommation

Approche 1

$$\sum_{i=s}^{n} f(i) = \sum_{i=s}^{j} f(i) + \sum_{i=j+1}^{n} f(i)$$

Approche 2

$$\sum_{i=s}^{n} f(i) = \sum_{i=0}^{n} f(i) - \sum_{i=0}^{s-1} f(i)$$

10.2.2 Sommations particulières

Sommation d'une constante

Soit $n \in \mathbb{N}$ et $c \in \mathbb{R}$, alors

$$\sum_{i=1}^{n} c = nc$$

Somme arithmétique

Soit $n \in \mathbb{N}$, alors la somme des n premiers entiers est

$$\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$$

Somme des carrés

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

Théorème de Nicomachus

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Somme géométrique

Soit $n \in \mathbb{N}$ et $a, r \in \mathbb{R}, r \neq 1$, alors

$$\sum_{i=0}^{n} ar^i = \frac{a(1-r^{n+1})}{1-r} = \frac{a(r^{n+1}-1)}{r-1}$$

Somme géométrique avec un multiplicateur variable

Soit $n \in \mathbb{N}$ et $r \in \mathbb{R}, r \neq 1$, alors

$$\sum_{i=0}^{n} ir^i = \frac{nr^{n+2} - (n+1)r^{n+1} + r}{(1-r)^2}$$

Somme télescopique

Soit $n \in \mathbb{N}$, alors

$$\sum_{i=0}^{n} (a_{i+1} - a_i) = a_{n+1} - a_0$$

10.2.3 Exercices

Exprimez les sommations suivantes sous la forme la plus simplifiée possible. Détaillez vos calculs.

Sommation 1

$$\sum_{i=k}^{m} \sum_{a=j}^{n} i \tag{10.1}$$

où k et j sont deux entiers tels que $k \leq m$ et $j \leq n$.

Sommation 2

$$\sum_{i=1}^{n} (2i-1) \tag{10.2}$$

Sommation 3

$$\sum_{j=0}^{n-1} j2^j \tag{10.3}$$

Sommation 4

$$\sum_{i=0}^{\beta}(i^2) - \sum_{j=0}^{\beta}(\beta - j) \tag{10.4}$$

Sommation 5

$$\sum_{d=n+1}^{n} d^4 \tag{10.5}$$

Sommation 6

$$\sum_{\alpha=1}^{k} \left(\frac{3}{2}\right)^{\alpha} \tag{10.6}$$

Sommation 7

$$\sum_{i=0}^{\log_3 n} \left(\frac{2}{3}\right)^{i} \tag{10.7}$$

Sommation 8

$$\sum_{i=0}^{\log_3 n} n\left(\frac{2}{3}\right)^{i} \tag{10.8}$$

Sommation 9

$$\sum_{k=2}^{p} k \tag{10.9}$$

Sommation 10

$$\sum_{p=0}^{n}(\sum_{p=0}^{n}5+\sum_{p=0}^{n}2) \tag{10.10}$$

Sommation 11

$$\sum_{\theta=0}^{n}(\theta-\sum_{\psi=0}^{m}2\psi) \tag{10.11}$$

Sommation 12

$$\sum_{i=1}^{\log n}\log(\frac{n}{2^i}) \tag{10.12}$$

Sommation 13

$$\sum_{s=0}^{n-1}s \tag{10.13}$$

Sommation 14

$$\sum_{k=2\sigma}^{m}k \tag{10.14}$$

où σ est un entier tel que $2\sigma \leq m$.

10.3 Démonstrations

10.3.1 Démonstrations générales

Exercice 1

Démontrez que $\sqrt{2}$ est un nombre irrationnel.

Exercice 2

Démontrez que

$$\sum_{i=0}^{n}i=\frac{n(n+1)}{2} \tag{10.15}$$

Note : la démonstration doit être faite sans utiliser le principe d'induction.

Exercice 3

Soit $k, s \in \mathbb{N}$, $f : \mathbb{N} \to \mathbb{R}$ et $b > 0, b \neq 1, b \in \mathbb{R}$, démontrez que

$$\sum_{k=s}^{n}(\log_b(f(k))) = \log_b(\prod_{k=s}^{n} f(k)) \tag{10.16}$$

Exercice 4

Démontrez que

$$4n^3 + n^2 \in \mathcal{O}(n^3) \tag{10.17}$$

Exercice 5

Démontrez que

$$\log n \notin \mathcal{O}(\log \log n) \tag{10.18}$$

10.3.2 Démonstrations par induction

Exercice 1

Démontrez que

$$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \tag{10.19}$$

Exercice 2

Démontrez que

$$\sum_{i=0}^{n} ar^i = \frac{a(r^{n+1}-1)}{r-1} \tag{10.20}$$

Exercice 3

Les nombres de Lucas sont des nombres qui suivent la récurrence suivante :

$$L_n = \begin{cases} 2 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ L_{n-1} + L_{n-2} & \text{sinon} \end{cases} \tag{10.21}$$

Démontrez que les nombres de Lucas respectent bel et bien cette récurrence.

Exercice 4

Soit la suite a_n définie ainsi :

$$a_n = \begin{cases} 1 & \text{si } n = 0 \\ 3 & \text{si } n = 1 \\ 2(a_{n-1}) + a_{n-2} & \text{sinon} \end{cases} \tag{10.22}$$

Démontrez que a_n est toujours un nombre impair.

Exercice 5

Démontrez que $(k+1)^3 - (k+1)$ est divisible par 3.

10.4 Corrigé

Série 1

Exercice 1

a) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C.

```
1  def additionner(A, B):
2      retenue = 0
3      C = [0] * (len(A) + 1)
4      i = 0
5      while i < len(A):
6          C[i] = (A[i] + B[i] + retenue) % 2
7          retenue = (A[i] + B[i] + retenue) // 2
8          i = i + 1
9      C[len(A)] = retenue
10     return C
```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière position des tableaux A, B et C.

```
1  def additionner(A, B):
2      retenue = 0
3      C = [0] * (len(A) + 1)
4      i = len(A) - 1
5      while i >= 0:
6          C[i + 1] = (A[i] + B[i] + retenue) % 2
7          retenue = (A[i] + B[i] + retenue) // 2
8          i = i - 1
9      C[0] = retenue
10     return C
```

b) Dans les deux cas, une opération barométrique identifiable est la comparaison effectuée à la ligne 5, qui s'effectue $n+1$ fois, où n est la taille des tableaux A et B. Puisque cette ligne s'exécute en temps constant à chaque fois, la complexité temporelle de la fonction ADDITIONNER est $\mathcal{O}(n)$.

Exercice 2

a) Le tableau C doit maintenant avoir $2n$ éléments.

b) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C.

```
1   def multiplier(A,B):
2       C = [0] * (len(A))
3       decalage = 0
4       while decalage < len(A):
5           temp = [0] * (len(A) + decalage)
6           j = 0
7           while j < len(A):
8               temp[j + decalage] = A[j] * B[decalage]
9               j = j + 1
10          C = add_bin(temp, C)
11          decalage = decalage + 1
12      return C
```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière case des tableaux A, B et C.

```
1   def multiplier(A,B):
2       C = [0] * (len(A))
3       decalage = 0
4       while decalage < len(A):
5           temp = [0] * (len(A) + decalage)
6           j = len(A) - 1
7           while j >= 0:
8               temp[len(temp) - decalage - (len(A) - j - 1) - 1] = A[j] * B[len(B) - decalage - 1]
9               j = j - 1
10          C = add_bin(temp, C)
11          decalage = decalage + 1
12      return C
```

c) Une opération barométrique possiblement identifiable pour évaluer la complexité de cet algorithme est l'instruction qui concerne l'addition de deux nombres binaires (ligne 10). Étant donné que le tableau C doit avoir $2n$ éléments à la fin de l'algorithme et que ce dernier commence avec n éléments, le coût de cette instruction est de $\sum_{i=n}^{2n-1}(i+1)$ si on inclut le coût de l'assignation.

$$
\begin{aligned}
\tau(\text{MULTIPLIER}) &= \sum_{i=n}^{2n-1}(i+1) \\
&= \sum_{i=0}^{\psi} i + \sum_{i=0}^{\psi} 1 - (\sum_{i=0}^{n-1} i + \sum_{i=0}^{n-1} 1) \qquad \text{posons } \psi = 2n-1 \\
&= \frac{\psi(\psi+1)}{2} + \psi + 1 - (\frac{n(n-1)}{2} + n) \\
&= \frac{4n^2-2n}{2} + 2n - 1 + 1 - \frac{n^2+n}{2} \\
&= \frac{4n^2-2n+4n-n^2-n}{2} \\
&= \frac{3}{2}n^2 + \frac{1}{2}n \in \mathcal{O}(n^2)
\end{aligned}
$$

Exercice 3

```python
def contient_cycle(L):
    lent = obtenir_tete(L)
    rapide = obtenir_tete(L)
    if rapide is None:
        return False
    cyclique = False
    while not (rapide is None) and not (obtenir_suivant(rapide) is None) and not cyclique:
        lent = obtenir_suivant(lent)
        rapide = obtenir_suivant(obtenir_suivant(rapide))
        if rapide == lent:
            cyclique = True
    return cyclique
```

Exercice 4

a) En premier lieu, voici le nombre de fois où chaque opération est exécutée, ainsi que son coût :

Numéro de la ligne	Nombre d'exécutions	Coût
1	1	$c_1 = k + 2$
2	1	$c_2 = n + 2$
3	1	c_3
4	$n + 1$	c_4
5	n	c_5
6	n	c_6
7	1	c_7
8	$k + 1$	c_8
9	k	c_9
10	k	c_{10}
11	1	c_{11}
12	$n + 1$	c_{12}
13	n	c_{13}
14	n	c_{14}
15	n	c_{15}
16	1	c_{16}

Le coût total de cette procédure est le suivant :

$$\begin{aligned}
\tau(\text{TRICOMPTAGE}) &= c_1 + c_2 + c_3 + \sum_{i=0}^{n} c_4 + \sum_{i=0}^{n-1}(c_5 + c_6) + c_7 \\
&+ \sum_{i=0}^{k} c_8 + \sum_{i=0}^{k-1}(c_9 + c_{10}) + c_{11} + \sum_{i=0}^{n} c_{12} \\
&+ \sum_{i=0}^{n-1}(c_{13} + c_{14} + c_{15}) + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15}) \\
&+ k(c_8 + c_9 + c_{10}) + c_1 + c_2 + c_3 + c_4 \\
&+ c_7 + c_8 + c_{11} + c_{12} + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15} + 1) \\
&+ k(c_8 + c_9 + c_{10} + 1) + c_3 + c_4 + c_7 + c_8 \\
&+ c_{11} + c_{12} + c_{16} + 4
\end{aligned}$$

Donc, la complexité temporelle est $\mathcal{O}(n + k)$.
Note : le dernier développement correspond au cas où l'on développe les coûts c_1 et c_2, qui sont les seuls à ne pas être constants.

b) La complexité spatiale de cet algorithme est $\mathcal{O}(n + k)$.

Exercice 5

a) L'algorithme transforme la matrice passée en paramètre en matrice triangulaire inférieure : pour chaque case de la partie triangulaire supérieure, il ajoute à la case correspondante de la partie triangulaire inférieure (de façon à peu près égale selon la ligne correspondante) la valeur déjà présente dans la case, plus la somme des éléments faisant partie de la partie triangulaire inférieure.

b) Une opération barométrique possible est l'instruction se trouvant à la ligne 7. En effet, la comparaison est effectuée $\sum_{i=0}^{n-1} \sum_{j=i+1}^{n} 1$ fois. La raison de ce choix est que, parmi toutes les instructions présentes dans la boucle principale entre les lignes 5 et 19, les instructions qui sont exécutées le plus souvent sont les instructions de comparaison des boucles aux lignes 7 et 14.

c)

$$\begin{aligned}
\tau(\text{WERK}) &= \sum_{i=0}^{n-1} \sum_{j=i+1}^{n} 1 \\
&= \sum_{i=0}^{n-1} (\sum_{j=0}^{n} 1 - \sum_{j=0}^{i} 1) \\
&= \sum_{i=0}^{n-1} ((n+1) - (i+1)) \\
&= \sum_{i=0}^{n-1} n - \sum_{i=0}^{n-1} i \\
&= n \times n - \frac{n(n-1)}{2} \\
&= \frac{n^2+n}{2} \in \mathcal{O}(n^2)
\end{aligned}$$

Exercice 6

Approche naïve

a)

```
1   def somme_diagonale(M):
2       somme = 0
3       i = 0
4       while i < len(M):
5           j = 0
6           while j < len(M):
7               if i == j or j == (len(M) - i - 1):
8                   somme = somme + M[i][j]
9               j = j + 1
10          i = i + 1
11      return somme
```

b) Une opération barométrique possible dans cet algorithme est l'instruction se situant à la ligne 6. En effet, l'instruction de comparaison effectuée à l'intérieur de cette boucle imbriquée s'exécute exactement $\sum_{i=0}^{n-1} \sum_{j=0}^{n} 1$ fois. Parmi toutes les instructions, c'est la seule qui s'exécute exactement $\sum_{i=0}^{n-1} \sum_{j=0}^{n} 1$ fois.

c)

$$\begin{aligned}
\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n} 1 \\
&= \sum_{i=0}^{n-1} (n+1) \\
&= (n+1) \sum_{i=0}^{n-1} 1 \\
&= (n+1)n \\
&= n^2 + n \in \mathcal{O}(n^2)
\end{aligned}$$

Approche plus efficace

a)

```
1   def somme_diagonale(M):
2       somme = 0
3       milieu = 0
4       if len(M) % 2 == 1:
5           milieu = len(M) // 2
6       i = 0
7       while i < len(M):
8           somme = somme + M[i][i]
9           i = i + 1
10      j = i - 1
11      i = 0
12      while i < len(M):
13          somme = somme + M[i][j - i]
14          i = i + 1
15      if milieu > 0:
16          somme = somme - M[milieu][milieu]
17      return somme
```

b) Un choix possible pour l'opération barométrique dans cet algorithme est l'instruction de comparaison effectuée dans la boucle de la ligne 7. En effet, hormis l'instruction de la ligne 12, c'est la seule instruction qui est exécutée exactement $n+1$ fois, où n représente la dimension de la matrice M. Les autres instructions sont exécutées en temps constant, ou bien n fois pour les instructions aux lignes 8 et 13, qui auraient pu être aussi des choix possibles.

c)

$$\begin{aligned}\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^{n+1} 1 \\ &= n + 2 \in \mathcal{O}(n)\end{aligned}$$

Extra

Exercice 1

a)

```
1   def contient_doublons(T):
2       T = sorted(T)            # Utilisation d'un tri en temps O(n log n)
3       i = 1
4       while i < len(T):
5           if T[i] == T[i - 1]:
6               return True
7           i = i + 1
8       return False
```

b) Une opération barométrique possible serait celle à la ligne 2, soit le tri effectué par la fonction.

c) Si la fonction de tri est un tri dont la complexité est garantie en $\mathcal{O}(n \log n)$, alors la complexité asymptotique de cet algorithme est $\mathcal{O}(n \log n)$.

Exercice 2

a)

```
1   def il_y_a_somme(T, n):
2       sorted(T)            # Utilisation d'un tri en temps O(n log n)
3       i = 0
4       j = len(T) - 1
5       while i < j:
6           if T[i] + T[j] == n:
7               return True
8           else if T[i] + T[j] < n:
9               i = i + 1
10          else:
11              j = j - 1
12      return False
```

b) Une opération barométrique possible serait celle à la ligne 2, soit le tri effectué par la fonction.

c) Si la fonction de tri est un tri dont la complexité est garantie en $\mathcal{O}(n \log n)$, alors la complexité asymptotique de cet algorithme est $\mathcal{O}(n \log n)$.

Exercice 3

Une opération barométrique possible est la comparaison à la ligne 7. Cette dernière est effectuée

$$\sum_{i=0}^{n-1} \sum_{j=0}^{\log_2(n)-1} \sum_{k=0}^{\log_2 n} 1$$

fois. La complexité temporelle est

$$\begin{aligned}
\sum_{i=0}^{n-1} \sum_{j=0}^{\log_2(n)-1} \sum_{k=0}^{\log_2 n} 1 &= \sum_{i=0}^{n-1} \sum_{j=0}^{\log_2(n)-1} \log_2(n) + 1 \\
&= \sum_{i=0}^{n-1} \log_2^2(n) + \log_2(n) \\
&= n \log_2^2(n) + n \log_2 n \in \mathcal{O}(n \log_2^2 n)
\end{aligned}$$

Exercice 4

Une opération barométrique possible est la comparaison à la ligne 4, qui s'exécute n fois. La complexité asymptotique est par conséquent $\mathcal{O}(n)$.

Exercice 5

Une opération barométrique possible est la comparaison à la ligne 6, qui s'exécute $n-1$ fois, mais coûte $n+1$, où n est la taille du tableau. La complexité asymptotique est par conséquent $\mathcal{O}(n^2)$.

Exercice 6

Une opération barométrique possible est la comparaison à la ligne 8, qui s'exécute n fois. La complexité asymptotique est par conséquent $\mathcal{O}(n)$.

Exercice 7

Approche naïve

```
1  def rotation_droite(T, k):
2      if len(T) > 0:
3          decalage = k % len(T)
4          temp = [0] * len(T)
5          i = 0
6                      while i < len(T) and not (decalage == 0) :
7              temp[(i + k) % len(T)] = T[(i + k) % len(T)]
8                    T = temp
```

Dans ce cas, les complexités temporelle et spatiale sont en $\mathcal{O}(n)$.

Approche plus efficace

```
1  # On évite de redéfinir la même fonction à chaque inversion d'une partie du tableau
2  def inverser(T, debut, fin):
3      while debut < fin:
4          arr[debut], arr[fin] = arr[fin], arr[debut]
5          debut = debut + 1
6          fin = fin - 1
7
8  def rotate_in_place(T, k):
9      if len(T) > 0:
10         decalage = k % len(T)
11         inverser(T, 0, len(T)- decalage -1)
12         inverser(T, len(T)- decalage, len(T)-1)
13         inverser(T, 0, len(T)-1)
```

Dans ce cas, la complexité temporelle est en $\mathcal{O}(n)$ et la complexité spatiale en $\mathcal{O}(1)$.

Série 2

Exercice 1

Pour démontrer que

$$f + g \in \Theta(\max(f, g))$$

il faut démontrer que $f + g \in \mathcal{O}(\max(f, g))$ et que $f + g \in \Omega(\max(f, g))$.

Démonstration de la borne supérieure

Démontrons que $f + g \in \mathcal{O}(\max(f, g))$. Pour pouvoir démontrer cette borne, il faut que la démonstration reste valide dans le cas où $\max(f(n), g(n)) = f(n)$

et dans le cas où $\max(f(n), g(n)) = g(n)$. Démontrons le premier cas.

$$\begin{aligned} f(n) + g(n) &\leq \max(f(n), g(n)) \\ &\leq f(n) && (\forall n, n \geq 0) \\ &\leq f(n) + f(n) && (\forall n, n \geq 0) \\ &\leq 2f(n) && (\forall n, n \geq 0) \end{aligned}$$

On trouve la paire suivante : $c = 2, n_0 = 0$ dans le cas où $\max(f(n), g(n)) = f(n)$. Démontrons le deuxième cas.

$$\begin{aligned} f(n) + g(n) &\leq \max(f(n), g(n)) \\ &\leq g(n) && (\forall n, n \geq 0) \\ &\leq g(n) + g(n) && (\forall n, n \geq 0) \\ &\leq 2g(n) && (\forall n, n \geq 0) \end{aligned}$$

La paire $c = 2, n_0 = 0$ valide le deuxième cas.

Donc, $f + g \in \mathcal{O}(\max(f, g))$.

Démonstration de la borne inférieure

Démontrons que $f + g \in \Omega(\max(f, g))$.

$$f(n) + g(n) \geq \max(f(n), g(n)) \qquad (\forall n, n \geq 0)$$

La paire $c = 1, n_0 = 0$ confirme que $f + g \in \Omega(\max(f, g))$.

En conclusion, $f + g \in \Theta(\max(f, g))$.

Exercice 2

Démonstration de la borne supérieure

$$\begin{aligned} (n + a)^b &\leq (n + a)^b && (\forall n, n \geq 0) \\ &\leq (n + n)^b && (\forall n, n \geq a) \\ &\leq (2n)^b && (\forall n, n \geq a) \\ &\leq 2^b n^b && (\forall n, n \geq a) \end{aligned}$$

La paire $c = 2^b$ et $n_0 = a$ montre que $(n + a)^b \in \mathcal{O}(n^b)$.

Démonstration de la borne inférieure

$$(n + a)^b \geq n^b \qquad (\forall n, n \geq 0)$$

La paire $c = 1$ et $n_0 = 0$ montre que $(n + a)^b \in \Omega(n^b)$.

Donc, $(n + a)^b \in \Theta(n^b)$.

Exercice 3

Note : afin d'éviter toute confusion, α représente la première constante dans le cas de la borne supérieure et β celle de la borne inférieure.

Démonstration de la borne supérieure

$$\begin{aligned}
\sum_{i=1}^{n} i^c &\leq \sum_{i=1}^{n} i^c && (\forall n, n \geq 0) \\
&\leq \sum_{i=1}^{n} n^c && (\forall n, n \geq 0) \\
&\leq n^c \times n && (\forall n, n \geq 0) \\
&\leq n^{c+1} && (\forall n, n \geq 0)
\end{aligned}$$

La paire $\alpha = 1$ et $n_0 = 0$ montre que $\sum_{i=1}^{n} i^c \in \mathcal{O}(n^{c+1})$.

Démonstration de la borne inférieure

$$\begin{aligned}
\sum_{i=1}^{n} i^c &\geq \sum_{i=1}^{n} i^c && (\forall n, n \geq 0) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \frac{n^c}{2^c} && (\forall n, n \geq 0) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left(\frac{n}{2}\right)^c && (\forall n, n \geq 0) \\
&\geq \left(\frac{n}{2}\right)^c \times \left\lfloor \frac{n}{2} \right\rfloor && (\forall n, n \geq 0) \\
&\geq \frac{1}{2^{c+1}} n^{c+1} && (\forall n, n \geq 0)
\end{aligned}$$

La paire $\beta = \frac{1}{2^{c+1}}$ et $n_0 = 0$ montre que $\sum_{i=1}^{n} i^c \in \Omega(n^{c+1})$.

En conclusion, $\sum_{i=1}^{n} i^c \in \Theta(n^{c+1}), c > -1$.

Exercice 4

Voici l'ordre des fonctions, par ordre de grandeur croissant :

$$\log^2 n, 2^{\log n}, \log n!, n^3, (\log n)^{\log n}, \left(\sqrt{n}\right)^{\log n}, \left(\frac{3}{2}\right)^n, 2^{n+1}, 2^{2n}, n!, 2^{2^n}$$

Exercice 5

a) Vrai, la constante multiplicative dans $2n$ ne change pas la complexité asymptotique dans ce contexte et donc, $2n \in \Theta(n)$.
b) Faux, avec la notation o, une fonction ne peut pas croître strictement plus vite qu'elle-même.
c) Faux, avec la notation ω, une fonction ne peut pas croître strictement plus vite qu'elle-même.
d) Faux, si on tente de démontrer que $n! \in \Omega((n+1)!)$, on tombe sur la contradiction suivante :

$$\begin{aligned} n! &\geq c(n+1)! \\ n! &\geq c(n+1)n! \\ 1 &\geq c(n+1) \end{aligned} \qquad \text{Contradiction}$$

Il n'existe pas de constantes c et n_0 telles que $n! \in \Omega((n+1)!)$. Donc, $n! \notin \Theta((n+1)!)$.
e) Vrai, la sommation de gauche se simplifie en n^2. Or, dans le cas Ω, la fonction peut croître plus vite ou à la même vitesse qu'elle-même.
f) Faux, dans le cas de fonctions exponentielles qui n'ont pas la même base, c'est celle qui a la plus grande base qui domine et donc $2^n \notin \Omega(3^n)$.
g) Faux, une fois la sommation simplifiée, le terme dominant est n^2, qui croît plus vite que n. Donc, $n^2 \notin \Theta(n)$.
h) Vrai, deux logarithmes de bases différentes croissent à la même vitesse. En effet, on peut changer la base d'un logarithme, ce qui ajoute simplement un facteur constant, sans effet dans ce contexte.

Exercice 6

a) Si $f(n) = 2n$ et $g(n) = n$, alors $f \in \mathcal{O}(g)$, mais $2^f \notin \mathcal{O}(2^g)$.
b) Si $f(n) = n^2$ et $g(n) = n$, alors $f + g \notin \Theta(\min(f, g))$, car $f(n)$ est le terme dominant, alors que $\min(f, g) = g$ ne croît pas à la même vitesse. En effet, $f \in \omega(g)$.
c) Si $f(n) = n^2$ et $g(n) = n$, alors $f \in \Omega(g)$, mais $g \notin \Omega(f)$, car $g \in o(f)$. En effet, la fonction g croît strictement moins vite que la fonction f.

Extra

Exercice 1

Pour démontrer que $\log(n!) \in \Theta(n \log n)$, il faut démontrer que $\log(n!) \in \mathcal{O}(n \log n)$ et que $\log(n!) \in \Omega(n \log n)$. Avant de démontrer une des bornes, on peut faire la simplification suivante : $\log(n!) = \sum_{i=1}^{n} \log(i)$.

Démonstration de la borne supérieure

Démontrons que $\log(n!) \in \mathcal{O}(n \log n)$.

$$\begin{aligned}
\sum_{i=1}^{n} \log i &\leq \sum_{i=1}^{n} \log i && (\forall n, n \geq 1) \\
&\leq \sum_{i=1}^{n} \log n && (\forall n, n \geq 1) \\
&\leq \log n \sum_{i=1}^{n} 1 && (\forall n, n \geq 1) \\
&\leq n \log n && (\forall n, n \geq 1)
\end{aligned}$$

Ici, $c = 1$ et $n_0 = 1$. Donc, $\log(n!) \in \mathcal{O}(n \log n)$.

Démonstration de la borne inférieure

Démontrons que $\log(n!) \in \Omega(n \log n)$.

$$\begin{aligned}
\sum_{i=1}^{n} \log i &\geq \sum_{i=1}^{n} \log i && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \log \frac{n}{2} && (\forall n, n \geq 1) \\
&\geq \log n \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 - \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 && (\forall n, n \geq 1) \\
&\geq \left\lfloor \frac{n}{2} \right\rfloor \log n - \left\lfloor \frac{n}{2} \right\rfloor && (\forall n, n \geq 1) \\
&\geq \frac{n}{2}(\log n - 1) && (\forall n, n \geq 1) \\
&\geq \frac{n}{2}(\frac{1}{2} \log n) && (\forall n, n \geq 4) \\
&\geq \frac{1}{4} n \log n && (\forall n, n \geq 4)
\end{aligned}$$

Ici, $c = \frac{1}{4}$ et $n_0 = 4$. Donc, $\log(n!) \in \Omega(n \log n)$.

En conclusion, $\log(n!) \in \Theta(n \log n)$.

Exercice 2

a) Pour démontrer cette propriété, il est possible d'utiliser la même approche qu'à l'exercice précédent.

Démonstration de la borne supérieure

$$\begin{aligned}
\sum_{i=1}^{n} \log^k i &\leq \sum_{i=1}^{n} \log^k i && (\forall n, n \geq 1) \\
&\leq \sum_{i=1}^{n} \log^k n && (\forall n, n \geq 1) \\
&\leq \log^k n \sum_{i=1}^{n} 1 && (\forall n, n \geq 1) \\
&\leq n \log^k n && (\forall n, n \geq 1)
\end{aligned}$$

On trouve comme paire de constantes $c = 1$ et $n_0 \geq 1$ qui satisfont l'inéquation. Donc, $\sum_{i=1}^{n} \log^k i \in \mathcal{O}(n \log^k i)$.

Démonstration de la borne inférieure

$$
\begin{aligned}
\sum_{i=1}^{n} \log^k i &\geq \sum_{i=1}^{n} \log^k i && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \log^k (\frac{n}{2}) && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (\log(n) - \log 2)^k && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (\log(n) - \log 2)^k && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (\log(n) - \log 2)^k && (\forall n, n \geq 1) \\
&\geq (\log(n) - 1)^k \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} 1 && (\forall n, n \geq 1) \\
&\geq \frac{n}{2} (\log(n) - 1)^k && (\forall n, n \geq 1) \\
&\geq \frac{n}{2} (\frac{1}{2} \log(n))^k && (\forall n, n \geq 4) \\
&\geq \frac{1}{2^{k+1}} n \log^k n && (\forall n, n \geq 4)
\end{aligned}
$$

On trouve comme paire de constantes $c = \frac{1}{2^{k+1}}$ et $n_0 \geq 4$ qui satisfont l'inéquation. Donc, $\sum_{i=1}^{n} \log^k i \in \Omega(n \log^k i)$. En conclusion, $\sum_{i=1}^{n} \log^k i \in \Theta(n \log^k n)$

b) Afin de démontrer cette propriété, il est utile de reprendre l'approche de la première partie de cet exercice ainsi que celle de l'exercice 3 de la série 2.

Démonstration de la borne supérieure

$$\begin{aligned}
\sum_{i=1}^{n} i^d \log^k i &\leq \sum_{i=1}^{n} i^d \log^k i && (\forall n, n \geq 1) \\
&\leq \sum_{i=1}^{n} n^d \log^k n && (\forall n, n \geq 1) \\
&\leq n^d \log^k n \sum_{i=1}^{n} 1 && (\forall n, n \geq 1) \\
&\leq n^{d+1} \log^k n && (\forall n, n \geq 1)
\end{aligned}$$

On trouve comme paire de constantes valides $c = 1$ et $n_0 = 1$. Donc, $\sum_{i=1}^{n} i^d \log^k i \in \mathcal{O}(n^{d+1} \log^k n)$.

Démonstration de la borne inférieure

$$\begin{aligned}
\sum_{i=1}^{n} i^d \log^k i &\geq \sum_{i=1}^{n} i^d \log^k i && (\forall n, n \geq 1) \\
&\geq \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} \left(\frac{n}{2}\right)^d \log^k (\frac{n}{2}) && (\forall n, n \geq 1) \\
&\geq \left(\frac{n}{2}\right)^d \log^k (\frac{n}{2}) && (\forall n, n \geq 1) \\
&\geq \frac{1}{2^{d+1}} n^{d+1} (\log^k n - 1) && (\forall n, n \geq 1) \\
&\geq \frac{1}{2^{d+1}} n^{d+1} \log^k (\sqrt{n}) && (\forall n, n \geq 4) \\
&\geq \frac{1}{2^{d+k+1}} n^{d+1} \log^k n && (\forall n, n \geq 4)
\end{aligned}$$

On trouve comme paire valide $c = \frac{1}{2^{d+k+1}}$ et $n_0 = 4$. Donc, $\sum_{i=1}^{n} i^d \log^k i \in \Omega(n^{d+1} \log^k n)$. En conclusion, $\sum_{i=1}^{n} i^d \log^k i \in \Theta(n^{d+1} \log^k n)$.

Exercice 3

Pour démontrer que $n^2 + n + \log n \notin \Omega(n^2 \log n)$, on va supposer par l'absurde qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telles que

$$n^2 + n + \log n \geq c n^2 \log n \quad (\forall n, n \geq n_0)$$

. Il se trouve que $n^2 + n + \log n \in \Theta(n^2)$. Donc, ceci est équivalent à tenter de démontrer que

$$n^2 \notin \Omega(n^2 \log n)$$

. On a alors

$$\begin{aligned} n^2 &\geq cn^2 \log n \\ \frac{n^2}{n^2} &\geq \frac{cn^2 \log n}{n^2} \\ 1 &\geq c \log n \\ \frac{1}{c} &\geq \log n \\ 2^{\frac{1}{c}} &\geq n \end{aligned}$$

Dès que $n_0 > 2^{\frac{1}{c}}$, l'inéquation est fausse et donc il n'existe pas de constantes c et n_0 satisfaisant l'inéquation ; ainsi, $n^2 + n + \log n \notin \Omega(n^2 \log n)$.

Exercice 4

Démontrons qu'il existe un $c > 0$ et un $n_0 \in \mathbb{N}$ tels que

$$n \geq c \log^2 n \quad (\forall n \geq n_0)$$

.

$$n \geq \log^2 n$$

Il se trouve qu'une paire valide est $c = 1$ et $n_0 \geq 16$. Donc, $n \in \Omega(\log^2 n)$.

Exercice 5

Démontrons que pour tout $c > 0$, il existe un $n_0 \in \mathbb{N}$ tel que

$$n^2 + 2n \leq cn^3 \quad (\forall n, n \geq n_0)$$

. Il se trouve que $n^2 + 2n \in \Theta(n^2)$.

$$\begin{aligned} n^2 + 2n &\leq n^2 + 2n && (\forall n, n \geq 0) \\ &\leq n^2 + 2n^2 && (\forall n, n \geq 0) \\ &\leq 3n^3 && (\forall n, n \geq 0) \end{aligned}$$

Dès que $n \geq \lceil \frac{3}{c} \rceil$, pour tout $c > 0$, l'inéquation $n^2 + 2n \leq cn^3$ est respectée et donc $n^2 + 2n \in o(n^3)$.

Exercice 6

Avant de démontrer cette propriété, il est utile de savoir que $\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \in \Theta(n^3)$. Donc, cela revient à démontrer que $n^3 + n \in \Theta(n^3)$. Pour démontrer que $n^3 + n \in \Theta(n^3)$, il faut démontrer que $n^3 + n \in \mathcal{O}(n^3)$ et $n^3 + n \in \Omega(n^3)$.

Démonstration de la borne supérieure

$$\begin{aligned} n^3 + n &\leq n^3 + n && (\forall n, n \geq 0) \\ &\leq n^3 + n^3 && (\forall n, n \geq 0) \\ &\leq 2n^3 && (\forall n, n \geq 0) \end{aligned}$$

Une paire qui convient est $c = 2$ et $n_0 = 0$. Donc, $n^3 + n \in \mathcal{O}(n^3)$.

Démonstration de la borne inférieure

$$\begin{aligned} n^3 + n &\geq n^3 + n && (\forall n, n \geq 0) \\ &\geq n^3 && (\forall n, n \geq 0) \end{aligned}$$

Une paire qui convient est $c = 1$ et $n_0 = 0$. Donc, $n^3 + n \in \Omega(n^3)$. En conclusion, $n^3 + n \in \Theta(n^3)$. Ainsi, $n^3 + n \in \Theta(\sum_{i=0}^{n} i^2)$.

Exercice 7

Si $k = 1$, alors elles croissent à la même vitesse. Dans le cas où $k > 1$, la fonction g peut être réécrite ainsi :

$$\begin{aligned} g(n) &= \log \log^k n \\ &= k \log \log n \end{aligned}$$

Il se trouve que dans ce cas, k n'apporte pas de changement asymptotique pour cette fonction, alors que, dans le cas de f, il a de l'importance. En effet, si on pose que $\eta = \log \log n$, alors les deux fonctions deviennent

$$\begin{aligned} f(\eta) &= \eta^k \\ g(\eta) &= k\eta \end{aligned}$$

On obtient donc, dans ce cas-ci, des polynômes dont le terme dominant est f, car k est utilisé comme exposant plutôt que comme coefficient multiplicatif de η. Donc, f croît plus vite que g si $k > 1$.

Exercice 8

On va supposer par l'absurde qu'il existe $n_0 \in \mathbb{N}$ tel que pour tout $c > 0$

$$n^3 + n \geq cn^3 \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^3 + n &\geq n^3 \\ &\geq n^3 + n^3 \\ &\geq 2n^3 \end{aligned}$$

contradiction

Dans le cas où $c = 2$, l'inéquation est fausse dès $n \geq 0$, ce qui contredit notre supposition de départ. Ainsi, la propriété de départ est vraie : $n^3 + n \notin \omega(n^3)$.

Exercice 9

$$\begin{aligned} 3^{2n} &\geq c3^n \\ \frac{3^{2n}}{3^n} &\geq c \\ 3^n &\geq c \end{aligned}$$

Une fonction exponentielle croît plus vite qu'une constante et finira tôt ou tard par dépasser n'importe quelle valeur assignée à c. Il existe donc $n_0 \in \mathbb{N}$ tel que $3^{2n} \geq c3^n$ pour tout $n \geq n_0$.

Exercice 10

Démontrer cette propriété revient à démontrer que $n! \notin \omega(n^n)$. Pour démontrer cette propriété, on va supposer par l'absurde que $n! \in \omega(n^n)$ et aussi que $n! \in \Omega(n^n)$ plutôt que de démontrer directement cette propriété. Dans ce cas-ci, il existe un $n_0 \in \mathbb{N}$ tel que pour tout $c > 0$

$$n! \geq cn^n \qquad (\forall n, n \geq n_0)$$

En tentant de démontrer cela, on aboutit au résultat suivant :

$$\begin{aligned} n! &\geq cn^n \\ \prod_{i=1}^{n} i &\geq c \prod_{i=1}^{n} n \\ \prod_{i=1}^{n} i &\geq c \prod_{i=1}^{n} n \end{aligned}$$

Il se trouve que dans le cas $c = 1$, dès que $n > 1$, l'inéquation est fausse. En réalité, peu importe le choix de valeur pour la constante c, l'inéquation finira par être fausse, car le produit de n par lui-même n fois domine le produit des entiers de 1 à n. Il s'ensuit que non seulement $n! \notin \omega(n^n)$, mais aussi que $n! \notin \Omega(n^n)$. Notre hypothèse de départ est donc contredite et ainsi, on a démontré que $n! \in o(n^n)$.

Note : au lieu d'écrire $1 \times 2 \times 3 \times \ldots \times (n-1) \times (n)$ et n^n de la même façon que la factorielle, les notations $\prod_{i=1}^{n} i$ et $\prod_{i=1}^{n} n$ sont utilisées à la place pour représenter les fonctions de manière plus claire, mais la première version aurait suffi.

Exercice 11

Pour démontrer cette propriété, on va supposer par l'absurde qu'il existe $n_0 \in \mathbb{N}$ tel que pour tout $c > 0$,

$$n^2 \leq cn \log n \qquad (\forall n, n \geq n_0)$$

En tentant de démontrer cette propriété, on tombe dans la contradiction suivante :

$$\begin{aligned} n^2 &\leq cn \log n \\ n &\leq \log n^c \\ 2^n &\leq n^c \end{aligned}$$

Dès que $c = 1$ et $n_0 > 0$, l'inéquation est fausse.

Ayant trouvé une contradiction dans notre supposition, on a confirmé que la propriété de départ était vraie. En conclusion, $n^2 \notin o(n \log n)$.

Exercice 12

Pour démontrer ce fait, on va supposer par l'absurde que $n \log n \in \Theta(n \log^2 n)$. Dans ce cas-ci, cela signifie que $n \log n \in \mathcal{O}(n \log^2 n)$ et $n \log n \in \Omega(n \log^2 n)$. Tentons de démontrer que $n \log n \in \Omega(n \log^2 n)$.

Dans ce cas, il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n \log n \geq cn \log^2 n \qquad (\forall n, n \geq n_0)$$

En tentant de démontrer cette propriété, on aboutit au résultat suivant :

$$\begin{aligned} n \log n &\geq cn \log^2 n \\ 1 &\geq c \log n \\ \frac{1}{c} &\geq \log n \end{aligned}$$

Il est impossible qu'une constante reste supérieure à une fonction croissante de n, ce qui contredit notre hypothèse de départ et confirme la propriété à démontrer. Ainsi, $n \log n \notin \Theta(n \log^2 n)$.

Exercice 13

Pour démontrer que $n \log n \in \Theta(n \log_9 n)$, il faut démontrer que $n \log n \in \mathcal{O}(n \log_9 n)$ et $n \log n \in \Omega(n \log_9 n)$.

Démonstration de la borne supérieure

Pour démontrer que $n \log n \in \mathcal{O}(n \log_9 n)$, il faut trouver une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n \log n \leq cn \log_9 n \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n \log n &\leq n \log n && (\forall n, n \geq 1) \\ &\leq \log_2(9) \left(\frac{1}{\log_2 9} n \log n \right) && (\forall n, n \geq 1) \\ &\leq \log_2(9) n \log_9 n && (\forall n, n \geq 1) \end{aligned}$$

Une paire valide est $c = \log_2 9$ et $n_0 = 1$. Donc, $n \log n \in \mathcal{O}(n \log_9 n)$.

Démonstration de la borne inférieure

Pour démontrer que $n \log n \in \Omega(n \log_9 n)$, il faut trouver une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n \log n \geq cn \log_9 n \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n \log n &\geq n \log n && (\forall n, n \geq 1) \\ &\geq \frac{1}{\log_9 2} n \log_9 n && (\forall n, n \geq 1) \end{aligned}$$

Une paire valide est $c = \frac{1}{\log_9 2}$ et $n_0 = 1$. Donc, $n \log n \in \Omega(n \log_9 n)$.

Exercice 14

Les deux fonctions croissent à la même vitesse. En effet, $\log n! \in \Theta(n \log n)$. Dans le cas de la fonction p, $\log n^n = n \log n \in \Theta(n \log n)$. Il s'agit donc du même ordre de croissance, exprimé sous une autre forme.

Exercice 15

Pour démontrer que $f_1 + f_2 \in \Theta(g)$, il faut démontrer que $f_1 + f_2 \in \mathcal{O}(g)$ et que $f_1 + f_2 \in \Omega(g)$.

Démonstration de la borne supérieure

Pour démontrer que $f_1 + f_2 \in \mathcal{O}(g)$, il faut trouver une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$f_1(n) + f_2(n) \leq cg(n) \qquad (\forall n, n \geq n_0)$$

Posons $c = 2a$, où a est une constante supérieure à 0.

$$\begin{aligned} f_1(n) + f_2(n) &\leq g(n) && (\forall n, n \geq n_0) \\ &\leq ag(n) + ag(n) && (\forall n, n \geq n_0) \\ &\leq 2ag(n) && (\forall n, n \geq n_0) \\ &\leq cg(n) && (\forall n, n \geq n_0) \end{aligned}$$

En réalité, puisque f_1 et f_2 sont du même ordre de grandeur que g, il existe une paire de constantes c et n_0 qui respecte les contraintes citées précédemment. En effet, f_1 ne domine pas f_2 en termes de comportement asymptotique, et inversement. Il existe donc des constantes c et n_0 qui satisfont l'inéquation. Ainsi, $f_1 + f_2 \in \mathcal{O}(g)$.

Démonstration de la borne inférieure

Pour démontrer que $f_1 + f_2 \in \Omega(g)$, il faut trouver une paire de constantes $c_2 > 0$ et $n_0 \in \mathbb{N}$ telle que

$$f_1(n) + f_2(n) \geq c_2 g(n) \qquad (\forall n, n \geq n_0)$$

Posons $c_2 = 2b$, où b est une constante supérieure à 0.

$$\begin{aligned} f_1(n) + f_2(n) &\geq g(n) && (\forall n, n \geq n_0) \\ &\geq bg(n) + bg(n) && (\forall n, n \geq n_0) \\ &\geq 2bg(n) && (\forall n, n \geq n_0) \\ &\geq c_2 g(n) && (\forall n, n \geq n_0) \end{aligned}$$

On peut obtenir la même conclusion que dans le cas de la borne supérieure. Donc, $f_1 + f_2 \in \Omega(g)$. Ainsi, $f_1 + f_2 \in \Theta(g)$.

Exercice 16

Pour démontrer que $\log(x^4 + x^2) \in \Theta(\log x)$, il faut démontrer que $\log(x^4 + x^2) \in \mathcal{O}(\log x)$ et $\log(x^4 + x^2) \in \Omega(\log x)$.

Démonstration de la borne supérieure

Pour démontrer que $\log(x^4 + x^2) \in \mathcal{O}(\log x)$, il faut trouver une paire de constantes $c > 0$ et $x_0 \in \mathbb{N}$ telle que

$$\log(x^4 + x^2) \leq c \log x \qquad (\forall x, x \geq x_0)$$

$$\begin{aligned}
\log(x^4 + x^2) &\leq \log(x^4 \times x^2) && (\forall x, x \geq 1)\\
&\leq \log(x^6) && (\forall x, x \geq 2)\\
&\leq 6\log(x) && (\forall x, x \geq 2)
\end{aligned}$$

On a comme paire valide $c = 6$ et $x_0 = 2$.

Démonstration de la borne inférieure

Pour démontrer que $\log(x^4 + x^2) \in \Omega(\log x)$, il faut trouver une paire de constantes $c > 0$ et $x_0 \in \mathbb{N}$ telle que

$$\log(x^4 + x^2) \geq c\log x \qquad (\forall x, x \geq x_0)$$

$$\begin{aligned}
\log(x^4 + x^2) &\geq \log(x^4 + x^2) && (\forall x, x \geq 1)\\
&\geq \log(x^4) && (\forall x, x \geq 1)\\
&\geq 4\log(x) && (\forall x, x \geq 1)
\end{aligned}$$

Les constantes $c = 4$ et $x_0 = 1$ satisfont l'inéquation. Donc, $\log(x^4 + x^2) \in \Omega(\log x)$. Ainsi, $\log(x^4 + x^2) \in \Theta(\log x)$.

Exercice 17

Pour prouver cette propriété, il faut donner $x_0 \in \mathbb{N}$ tel que pour tout $c > 0$

$$\left\lfloor \frac{x}{2} \right\rfloor \geq c\log x \qquad (\forall x, x \geq x_0)$$

$$\begin{aligned}
\left\lfloor \frac{x}{2} \right\rfloor &\geq c\log x\\
\left\lfloor \frac{x}{2} \right\rfloor &\geq \log x^c\\
2^{\left\lfloor \frac{x}{2} \right\rfloor} &\geq x^c
\end{aligned}$$

Dans ce cas, peu importe la valeur de c, il existera toujours un n_0 tel que l'inéquation restera vraie pour tout $n \geq n_0$. En effet, une fonction exponentielle croît plus vite qu'un polynôme. Donc, $\left\lfloor \frac{x}{2} \right\rfloor \in \omega(\log x)$.

Exercice 18

Pour prouver cette propriété, il faut donner $n_0 \in \mathbb{N}$ tel que pour tout $c > 0$

$$7n^4 + 3 \geq cn^3 \qquad (\forall n, n \geq n_0)$$

$$7n^4 + 3 \geq cn^3$$
$$7n + \frac{3}{n^3} \geq c$$

Peu importe la valeur de c, la partie de gauche de l'inéquation finira toujours par dépasser c. Donc, il existe un $n_0 \in N$ tel que pour tout $c > 0$, $7n^4 + 3 \geq cn^3$ pour tout $n \geq n_0$. Donc, $7n^4 + 3 \in \omega(n^3)$.

Exercice 19

Pour déterminer quelle fonction croît le plus vite, on va essayer de prouver que $f \in o(g)$ dès que $k \geq 2$. Le but est donc de déterminer s'il existe un $n_0 \in \mathbb{N}$ tel que, pour tout $c > 0$,

$$^nk \leq {}^{(n+1)}kc \qquad (\forall n, n \geq n_0)$$

Dans le cas où $k \geq 2$, la preuve est la suivante :

$$\begin{aligned} ^nk &\leq {}^{(n+1)}kc \\ m &\leq ck^m && \text{posons } m = {}^nk \\ \frac{m}{k^m} &\leq c \end{aligned}$$

Dans ce cas, même après substitution de variable, en faisant varier m, et ce, peu importe la valeur de $k \geq 2$, la partie de gauche ne fait que décroître par rapport à la partie de droite de l'inéquation. Cette décroissance peut être comparée à $\frac{1}{k}$, qui donne une plus grande valeur, mais décroît à la même vitesse que la partie de gauche. Donc, plus généralement, il existe un m tel que $\frac{m}{k^m} \leq \frac{1}{k} \leq c$ pour tout $c > 0$. Ainsi, peu importe la valeur de $c > 0$, il existe bel et bien un n_0 valide, ce qui confirme que $^nk \in o(^{n+1}k)$. Donc, la fonction g croît plus vite que la fonction f si $k \geq 2$.

Exercice 20

Pour démontrer cette propriété, il faut démontrer que $n^2 \log^2(n^2 + n) \in \mathcal{O}(n^2 \log^2 n)$ et $n^2 \log^2(n^2 + n) \in \Omega(n^2 \log^2 n)$.

Démonstration de la borne supérieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^2 \log^2(n^2 + n) \leq cn^2 \log^2 n \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^2 \log^2(n^2+n) &\leq n^2 \log^2(n^2+n) && (\forall n, n \geq 1) \\ &\leq n^2 \log^2(n^2 \times n) && (\forall n, n \geq 2) \\ &\leq n^2 \log^2(n^3) && (\forall n, n \geq 2) \\ &\leq 9n^2 \log^2 n && (\forall n, n \geq 2) \end{aligned}$$

On a comme paire valide de constantes $c = 9$ et $n_0 = 2$. Donc, $n^2 \log^2(n^2+n) \in \mathcal{O}(n^2 \log^2(n))$.

Démonstration de la borne inférieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^2 \log^2(n^2+n) \geq cn^2 \log^2 n \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^2 \log^2(n^2+n) &\geq n^2 \log^2(n^2+n) && (\forall n, n \geq 1) \\ &\geq n^2 \log^2 n && (\forall n, n \geq 1) \end{aligned}$$

On a comme paire valide de constantes $c = 1$ et $n_0 = 1$. Donc, $n^2 \log^2(n^2+n) \in \Omega(n^2 \log^2(n))$. Ainsi, $n^2 \log^2(n^2+n) \in \Theta(n^2 \log^2 n)$.

Exercice 21

Pour démontrer que $n^3 + n \in \Theta(n^3)$, il faut démontrer que $n^3 + n \in \mathcal{O}(n^3)$ et $n^3 + n \in \Omega(n^3)$.

Démonstration de la borne supérieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^3 + n \leq cn^3 \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^3 + n &\leq n^3 + n && (\forall n, n \geq 0) \\ &\leq n^3 + n^3 && (\forall n, n \geq 0) \\ &\leq 2n^3 && (\forall n, n \geq 0) \end{aligned}$$

Une paire de constantes valide est $c = 2$ et $n_0 = 0$. Donc, $n^3 + n \in \mathcal{O}(n^3)$.

Démonstration de la borne inférieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^3 + n \geq cn^3 \qquad (\forall n, n \geq n_0)$$

$$n^3 + n \geq n^3 \qquad (\forall n, n \geq 0)$$

Une paire de constantes valide est $c = 1$ et $n_0 \geq 0$. Donc, $n^3 + n \in \Omega(n^3)$. Ainsi, les deux bornes ayant été démontrées, $n^3 + n \in \Theta(n^3)$.

Exercice 22

Pour démontrer que $n^2 - n \in \omega(n)$, il faut trouver $n_0 \in \mathbb{N}$ tel que pour tout $c > 0$

$$n^2 - n \geq cn \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^2 - n &\geq cn \\ n - 1 &\geq c \end{aligned}$$

Dans ce cas-ci, peu importe le choix de la constante c, il existera toujours une valeur pour n_0 telle que $n^2 - n \geq cn$ pour tout $n \geq n_0$. En effet, une constante finit toujours par être dépassée par une fonction croissante non bornée. Donc, $n^2 - n \in \omega(n)$.

Exercice 23

Pour démontrer que $n - 5 \in \Theta(n)$, il faut démontrer que $n - 5 \in \mathcal{O}(n)$ et $n - 5 \in \Omega(n)$.

Démonstration de la borne supérieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n - 5 \leq cn \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n - 5 &\leq n - 5 && (\forall n, n \geq 0) \\ &\leq n + 5n && (\forall n, n \geq 1) \\ &\leq 6n && (\forall n, n \geq 1) \end{aligned}$$

Une paire de constantes valide est $c = 6$ et $n_0 = 1$. Donc, $n - 5 \in (n)$.

Démonstration de la borne inférieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n - 5 \geq cn \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n-5 &\geq n-5 && (\forall n, n \geq 0) \\ &\geq n && (\forall n, n \geq 0) \end{aligned}$$

Une paire de constantes valide est $c = 1$ et $n_0 = 0$. Donc, $n - 5 \in \Omega(n)$. Ainsi, les deux bornes ayant été démontrées, $n - 5 \in \Theta(n)$.

Exercice 24

Pour démontrer que $n^3+2n^2-n \in \Theta(n^3)$, il faut démontrer que $n^3+2n^2-n \in \mathcal{O}(n^3)$ et $n^3 + 2n^2 - n \in \Omega(n^3)$.

Démonstration de la borne supérieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^3 + 2n^2 - n \leq cn^3 \qquad (\forall n, n \geq n_0)$$

$$\begin{aligned} n^3 + 2n^2 - n &\leq n^3 + 2n^2 - n && (\forall n, n \geq 0) \\ &\leq n^3 + 2n^2 && (\forall n, n \geq 0) \\ &\leq n^3 + 2n^3 && (\forall n, n \geq 0) \\ &\leq 3n^3 && (\forall n, n \geq 0) \end{aligned}$$

Une paire de constantes valide est $c = 3$ et $n_0 = 0$. Donc, $n^3 + 2n^2 - n \in \mathcal{O}(n^3)$.

Démonstration de la borne inférieure

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$n^3 + 2n^2 - n \geq cn^3 \qquad (\forall n, n \geq n_0)$$

$$n^3 + 2n^2 - n \geq n^3 \qquad (\forall n, n \geq 0)$$

Une paire de constantes valide est $c = 1$ et $n_0 = 0$. Donc, $n^3 + 2n^2 - n \in \Omega(n^3)$. Ainsi, les deux bornes ayant été démontrées, $n^3 + 2n^2 - n \in \Theta(n^3)$.

Exercice 25

Démontrons qu'il existe une paire de constantes $c > 0$ et $n_0 \in \mathbb{N}$ telle que

$$4n^3 - 2n^2 - n \leq cn^3 \qquad (\forall n, n \geq n_0)$$

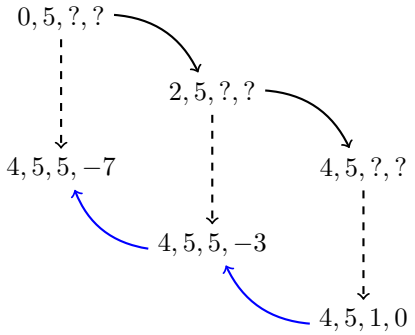
$$\begin{aligned} 4n^3 - 2n^2 - n &\leq 4n^3 - 2n^2 - n && (\forall n, n \geq 0) \\ &\leq 4n^3 && (\forall n, n \geq 0) \end{aligned}$$

Une paire valide est $c = 4$ et $n_0 = 0$. Donc, $4n^3 - 2n^2 - n \in \mathcal{O}(n^3)$.

Série 3

Exercice 1

a) Chaque nœud d'appel est noté sous la forme B_I, B_S, n_1, n_2.



b) BIDON() retourne le plus grand nombre entier du tableau (retourné dans le paramètre $n1$) et le plus petit nombre entier du tableau (retourné dans le paramètre $n2$).

c) $\tau(n) = \tau(n-1) + 3$ avec comme cas de base $\tau(2) = 1$.

d) En déroulant l'équation de récurrence, on a le résultat suivant :

$$\begin{aligned} \tau(n) &= \tau(n-2) + 3 \\ &= \tau(n-4) + 3 + 3 \\ &= \tau(n-6) + 3 + 3 + 3 \\ &\vdots \\ &= 1 + 3 + 3 + \ldots + 3 + 3 \end{aligned}$$

Sachant qu'on s'arrête à 2 et qu'on réduit la taille du problème de deux

à chaque appel, on a

$$\begin{aligned}\tau(n) &= \sum_{i=1}^{\frac{n-2}{2}} 3 + 1 \\ &= \frac{3n}{2} - 2 \in \Theta(n)\end{aligned}$$

Exercice 2

a) $\tau(n) = 2\tau(n-1) + 1$ avec comme cas de base $\tau(1) = 1$.
b) En déroulant l'équation de récurrence, on a le résultat suivant :

$$\begin{aligned}\tau(n) &= 2(2\tau(n-2)+1)+1 \\ &= 4(2\tau(n-3)+1)+2+1 \\ &= 8(2\tau(n-4)+1)+4+2+1 \\ &\vdots \\ &= 1+2+4+8+16+\ldots+2^{n-2}+2^{n-1}\end{aligned}$$

Il est possible de reformuler cette équation sous la forme fermée suivante :

$$\begin{aligned}\tau(n) &= \sum_{i=0}^{n-1}(2^i) \\ &= \frac{1-2^n}{1-2} \\ &= 2^n - 1 \in \Theta(2^n)\end{aligned}$$

Exercice 3

a)

```
1  def max_global(M, i):
2      if i >= len(M):
3          return M[i - 1][0]
4      j = 1
5      mx_local = M[i][0]
6      while j < len(M[i]):
7          mx_local = max(mx_local, M[i][j])
8          j = j + 1
9      return max(mx_local, max_global(M, i + 1))
```

b) L'équation de récurrence de cet algorithme est $\tau(n) = \tau(n-1) + (m-1)$ avec comme cas de base $\tau(0) = 1$, où n représente le nombre de tableaux dans la matrice et m la taille d'un tableau dans la matrice. Note : pour la construction de l'équation de récurrence, le $m-1$ vient de l'opération barométrique à la ligne 7.

c) En déroulant l'équation de récurrence, on a le développement suivant :

$$\begin{aligned}
\tau(n) &= \tau(n-1) + (m-1) \\
\tau(n) &= \tau(n-2) + (m-1) + (m-1) \\
\tau(n) &= \tau(n-3) + (m-1) + (m-1) + (m-1) \\
&\vdots \\
\tau(n) &= 1 + (m-1) + (m-1) + \ldots + (m-1) + (m-1)
\end{aligned}$$

Ce qui donne la forme fermée suivante :

$$\begin{aligned}
\tau(n) &= \sum_{i=1}^{n} (m-1) + 1 \\
&= (m-1) \sum_{i=1}^{n} 1 + 1 \\
&= (m-1) \times n + 1 \\
&= mn - n + 1
\end{aligned}$$

Exercice 4

```
1  def racine_entiere(n, bi, bs):
2      if bs == bi + 1:
3          return bi
4      milieu = (bi + bs) // 2
5      if milieu * milieu > n:
6          return racine_entiere(n, bi, milieu)
7      return racine_entiere(n, milieu, bs)
```

Exercice 5

a) $\tau \in \Theta(n^3)$
b) $\tau \in \Theta(n^{\log_3 4})$
c) $\tau \in \Theta(n!)$
d) $\tau \in \Theta(n^3 \log n)$
e) $\tau \in \Theta(2^n)$
f) $\tau \in \Theta(n^{\log_3 2})$
g) $\tau \in \Theta(n^{\log_3 4} \log n)$
h) $\tau \in \Theta(n^2 \log n^3)$
i) $\tau \in \Theta(n^{\frac{3}{2}})$

Exercice 6

En utilisant n'importe quelle méthode permettant de résoudre une équation de récurrence aux divisions finies, la complexité asymptotique de τ_A est $\Theta(n^{\log_2 7})$.

Puisque $\tau'_A \in o(\tau_A)$, on sait que $c < 4^{\log_2 7}$. Si $c = 4^{\log_2 7}$, alors $\tau'_A \notin o(\tau_A)$. Pour trouver la plus grande valeur possible pour c, il faut résoudre l'inéquation

précisée précédemment. En résolvant l'inéquation, on trouve la valeur suivante :

$$\begin{aligned} c &< 4^{\log_2 7} \\ c &< \left(2^{\log_2 7}\right)^2 \\ c &< 7^2 \\ c &< 49 \end{aligned}$$

En prenant en compte l'inégalité stricte, on obtient $c = 48$.

Exercice 7

Pour cette section, les formes fermées peuvent être obtenues en déroulant l'équation correspondante ou en dessinant l'arbre des appels.

a)

$$\begin{aligned} \tau(n) &= \sum_{i=0}^{\log_3 n} 2^i \left(\frac{n}{3^i}\right) \\ &= \sum_{i=0}^{p} 2^i \left(\frac{3^p}{3^i}\right) && \text{posons } n = 3^p \\ &= 3^p \sum_{i=0}^{p} \left(\frac{2}{3}\right)^i \\ &= 3^p \left(\frac{1 - \left(\frac{2}{3}\right)^{p+1}}{1 - \frac{2}{3}}\right) \\ &= 3^p \left(\frac{1 - \left(\frac{2}{3}\right)^{p+1}}{\frac{1}{3}}\right) \\ &= 3^p \left(3 - 2\left(\frac{2}{3}\right)^p\right) \\ &= n \left(3 - 2\left(\frac{2}{3}\right)^{\log_3 n}\right) \\ &= n \left(3 - 2n^{\log_3(\frac{2}{3})}\right) \\ &= 3n - 2n^{\log_3 2} \end{aligned}$$

b)

$$\begin{aligned}
\tau(n) &= \sum_{i=0}^{\log_2 n} 4^i \sqrt{\frac{n}{2^i}} \\
&= \sum_{i=0}^{p} 4^i \sqrt{\frac{2^p}{2^i}} && \text{posons } n = 2^p \\
&= 2^{\frac{p}{2}} \sum_{i=0}^{p} \left(\frac{4}{\sqrt{2}}\right)^i \\
&= 2^{\frac{p}{2}} \left(\frac{1 - \frac{4}{\sqrt{2}}^{p+1}}{1 - \frac{4}{\sqrt{2}}}\right) \\
&= 2^{\frac{p}{2}} \left(\frac{1}{1 - \frac{4}{\sqrt{2}}} - \frac{\frac{4}{\sqrt{2}}}{1 - \frac{4}{\sqrt{2}}} \left(\frac{4}{\sqrt{2}}\right)^p\right) \\
&= \sqrt{n} \left(\frac{1}{1 - \frac{4}{\sqrt{2}}} - \frac{\frac{4}{\sqrt{2}}}{1 - \frac{4}{\sqrt{2}}} \left(\frac{4}{\sqrt{2}}\right)^{\log_2 n}\right) \\
&= \sqrt{n} \left(\frac{1}{1 - \frac{4}{\sqrt{2}}} - \frac{\frac{4}{\sqrt{2}}}{1 - \frac{4}{\sqrt{2}}} n^{\log_2 \frac{4}{\sqrt{2}}}\right) \\
&= \frac{1}{1 - \frac{4}{\sqrt{2}}} \sqrt{n} - \frac{\frac{4}{\sqrt{2}}}{1 - \frac{4}{\sqrt{2}}} n^2 \in \Theta(n^2)
\end{aligned}$$

c)

$$\begin{aligned}
\tau(n) &= \sum_{i=0}^{\log_2 n} 2^i \left(\frac{n}{2^i}\right) \\
&= \sum_{i=0}^{p} 2^i \left(\frac{2^p}{2^i}\right) && \text{posons } n = 2^p \\
&= 2^p \sum_{i=0}^{p} 1 \\
&= 2^p (p+1) \\
&= n \log_2 n + n \in \Theta(n \log n)
\end{aligned}$$

d)

$$
\begin{aligned}
\tau(n) &= \sum_{i=0}^{\log_3 n} 2^i \left(\log \frac{n}{3^i}\right) \\
&= \sum_{i=0}^{p} 2^i \left(\log \frac{3^p}{3^i}\right) && \text{posons } n = 3^p \\
&= \sum_{i=0}^{p} 2^i \log 3^p - \sum_{i=1}^{p} 2^i \log 3^i && \text{posons } n = 3^p \\
&= \log 3^p \sum_{i=0}^{p} 2^i - \log 3 \sum_{i=1}^{p} i2^i \\
&= \log 3^p \left(\frac{1-2^{p+1}}{1-2}\right) - \log 3 \left(\frac{p2^{p+2} - (p+1)2^{p+1} + 2}{(1-2)^2}\right) \\
&= \log 3^p \left(2^{p+1} - 1\right) - \log 3 \left(2^p(p2^2 - (p+1)2^1) + 2\right) \\
&= \log 3^p \left(2^{p+1} - 1\right) - \log 3 \left(2^p(4p - 2p - 2) + 2\right) \\
&= \log 3^p \left(2^{p+1} - 1\right) - \log 3 (2^{p+1}(p-1) + 2) \\
&= \log n \left(2 \times 2^{\log_3 n} - 1\right) - \log 3 (2^{\log_3 n + 1}(\log_3 n - 1) + 2) \\
&= 2\log(n)2^{\log_3 n} - \log n - (2\log(3)\log_3 n 2^{\log_3 n} - 2\log(3)2^{\log_3 n} + 2\log 3) \\
&= 2\log(n)n^{\log_3 2} - \log n - (2\log_2(n)n^{\log_3 2} - 2\log(3)n^{\log_3 2} + 2\log 3) \\
&= 2\log(3)n^{\log_3 2} - \log n - 2\log 3 \in \Theta(n^{\log_3 2})
\end{aligned}
$$

e)

$$
\begin{aligned}
\tau(n) &= \sum_{i=0}^{\log_4 n} 4^i \left(\frac{n}{4^i}\log\frac{n}{4^i}\right) \\
&= \sum_{i=0}^{p} 4^i \left(\frac{4^p}{4^i}\log\frac{4^p}{4^i}\right) && \text{posons } n = 4^p \\
&= \sum_{i=0}^{p} 4^p \log\frac{4^p}{4^i} \\
&= 4^p \log 4^p \sum_{i=0}^{p} 1 - 2\times 4^p \sum_{i=1}^{p} i \\
&= 4^p \log 4^p (p+1) - 2\times 4^p \left(\frac{p(p+1)}{2}\right) \\
&= n\log n(\log_4 n + 1) - 2n\left(\frac{\log_4^2 n + \log_4 n}{2}\right) \\
&= n\log n\left(\frac{\log_2 n}{\log_2 4} + 1\right) - n\left(\frac{\log_2 n}{\log_2 4}\right)^2 - n\left(\frac{\log_2 n}{\log_2 4}\right) \\
&= \frac{1}{2}n\log^2 n + n\log n - \frac{1}{4}n\log^2 n - \frac{1}{2}n\log_2 n \\
&= \frac{1}{4}n\log_2^2 n + \frac{1}{2}n\log_2 n \in \Theta(n\log^2 n)
\end{aligned}
$$

Extra

Exercice 1

La complexité asymptotique de cette équation est $\Theta(\log\log n)$.

Exercice 2

```
1  def point_fixe (tab, bi, bs):
2      if bs < bi:
3          return False
4      milieu = (bi + bs) // 2
5      if tab[milieu] == milieu:
6          return True
7      elif tab[milieu] < milieu:
8          return point_fixe(tab, milieu + 1, bs)
9      else:
10         return point_fixe(tab, bi, milieu - 1)
```

Exercice 3

a) $\tau \in \Theta(n^{\log_5 3})$
b) $\tau \in \Theta(n^3)$
c) $\tau \in \Theta(n^3)$
d) $\tau \in \Theta(\log^5 n)$
e) $\tau \in \Theta(2^n)$
f) $\tau \in \Theta(n^{\log_4 7})$
g) $\tau \in \Theta(3^n)$
h) $\tau \in \Theta(n^{\frac{5}{2}})$
i) $\tau \in \Theta(n^2)$
j) $\tau \in \Theta(n^{\log_3 5})$
k) $\tau \in \Theta(4^n)$
l) $\tau \in \Theta(n)$
m) $\tau \in \Theta(n^3)$
n) $\tau \in \Theta(n^3)$
o) $\tau \in \Theta(n^2 \log n)$
p) $\tau \in \Theta(n \log n)$
q) $\tau \in \Theta(n^3)$
r) $\tau \in \Theta(n!)$

Exercice 4

a) Dans ce cas, en déroulant l'équation de récurrence avec $\beta = 1$ et $\tau(1) = \epsilon$, une constante, on a le résultat suivant :

$$\begin{aligned}
\tau(n) &= \tau(n-1) + \tau(1) + n \\
&= \tau(n-2) + \tau(1) + (n-1) + \tau(1) + n \\
&= \tau(n-3) + \tau(1) + (n-2) + \tau(1) + (n-1) + \tau(1) + n \\
&\vdots \\
&= \tau(1) + \tau(1) + 2 + \tau(1) + 3 + \ldots + \tau(1) + (n-1) + \tau(1) + n
\end{aligned}$$

Il est possible d'obtenir la forme fermée suivante :

$$\begin{aligned}
\tau(n) &= \sum_{i=2}^{n} i + n\tau(1) \\
&= \frac{n(n+1)}{2} - 1 + n\epsilon
\end{aligned}$$

b) En gardant la même valeur pour $\tau(1)$ et en posant $\tau(2) = \psi$, une constante, et $\beta = 2$, on a le déroulement suivant si n est pair :

$$\begin{aligned}
\tau(n) &= \tau(n-2) + \tau(2) + n \\
&= \tau(n-4) + \tau(2) + (n-1) + \tau(2) + n \\
&= \tau(n-6) + \tau(2) + (n-4) + \tau(2) + (n-2) + \tau(2) + n \\
&\vdots \\
&= \tau(2) + \tau(2) + 4 + \tau(2) + 6 + \ldots + \tau(2) + (n-4) + \tau(2) + (n-2) + \tau(2) + n
\end{aligned}$$

On peut obtenir la forme fermée suivante :

$$\begin{aligned}
\tau(n) &= \sum_{i=2}^{\frac{n}{2}} 2i + \frac{n}{2}\tau(2) \\
&= \frac{n^2}{4} + \frac{n}{2} - 2 + \frac{n\psi}{2}
\end{aligned}$$

Dans le cas où n est impair, on a le déroulement suivant :

$$\begin{aligned}
\tau(n) &= \tau(n-2) + \tau(2) + n \\
&= \tau(n-4) + \tau(2) + (n-1) + \tau(2) + n \\
&= \tau(n-6) + \tau(2) + (n-4) + \tau(2) + (n-2) + \tau(2) + n \\
&\vdots \\
&= \tau(1) + \tau(2) + 3 + \tau(2) + 5 + \ldots + \tau(2) + (n-4) + \tau(2) + (n-2) + \tau(2) + n
\end{aligned}$$

On peut obtenir la forme fermée suivante :

$$\begin{aligned}
\tau(n) &= \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} (2i+1) + \tau(1) + \left\lfloor \frac{n}{2} \right\rfloor \tau(2) \\
&= \left\lfloor \frac{n}{2} \right\rfloor \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \Big\rfloor + \left\lfloor \frac{n}{2} \right\rfloor + \epsilon + \psi \left\lfloor \frac{n}{2} \right\rfloor
\end{aligned}$$

c) La complexité asymptotique sera $\Theta(n^2)$.

Annexe

Sommation

Sommation 1

$$
\begin{aligned}
\sum_{i=k}^{m}\sum_{a=j}^{n} i &= \sum_{i=k}^{m} i \sum_{a=j}^{n} 1 \\
&= \sum_{i=k}^{m} i(\sum_{a=0}^{n} 1 - \sum_{a=0}^{j-1} 1) \\
&= \sum_{i=k}^{m} i(n+1-j) \\
&= (n+1-j)\sum_{i=k}^{m} i \\
&= (n+1-j)\left(\sum_{i=0}^{m} i - \sum_{i=0}^{k-1} i\right) \\
&= (n+1-j)\left(\frac{m(m+1)}{2} - \frac{k(k-1)}{2}\right) \\
&= (n+1-j)\left(\frac{m^2+m}{2} - \frac{k^2-k}{2}\right) \\
&= (n+1-j)\left(\frac{m^2+m-k^2+k}{2}\right) \\
&= (n+1-j)\left(\frac{(m+k)(m-k+1)}{2}\right) \\
&= \frac{(n+1-j)(m+k)(m-k+1)}{2}
\end{aligned}
$$

Sommation 2

$$\begin{aligned}\sum_{i=1}^{n}(2i-1) &= \sum_{i=1}^{n}2i - \sum_{i=1}^{n}1 \\ &= 2\sum_{i=1}^{n}i - \sum_{i=1}^{n}1 \\ &= 2\left(\frac{n(n+1)}{2}\right) - n \\ &= n^2 + n - n \\ &= n^2\end{aligned}$$

Sommation 3

$$\begin{aligned}\sum_{j=0}^{n-1}j2^j &= \sum_{j=0}^{n}(j2^j) - n2^n \\ &= \frac{n2^{n+2} - (n+1)2^{n+1} + 2}{(1-2)^2} - n2^n \\ &= n2^{n+2} - (n+1)2^{n+1} + 2 - n2^n \\ &= n2^{n+2} - n2^{n+1} - 2^{n+1} + 2 - n2^n \\ &= 2 + 2^n(4n - 2n - 2 - n) \\ &= (n-2)2^n + 2\end{aligned}$$

Sommation 4

$$\begin{aligned}\sum_{i=0}^{\beta}(i^2) - \sum_{j=0}^{\beta}(\beta - j) &= \sum_{i=0}^{\beta}(i^2) - (\sum_{j=0}^{\beta}\beta - \sum_{j=0}^{\beta}j) \\ &= \frac{\beta(\beta+1)(2\beta+1)}{6} - \left(\beta(\beta+1) - \frac{\beta(\beta+1)}{2}\right) \\ &= \frac{\beta(\beta+1)(2\beta+1)}{6} - \frac{\beta(\beta+1)}{2} \\ &= \frac{\beta(\beta+1)(2\beta+1) - 3\beta(\beta+1)}{6} \\ &= \frac{4n^3 - 2n}{6} \\ &= \frac{2n(2n^2+1)}{6}\end{aligned}$$

Sommation 5

$$\sum_{d=n+1}^{n} d^4 = 0$$

Sommation 6

$$\begin{aligned}\sum_{\alpha=1}^{k} \left(\frac{3}{2}\right)^{\alpha} &= \frac{1-(\frac{3}{2})^{k+1}}{1-\frac{3}{2}} \\ &= \frac{1-(\frac{3}{2})^{k+1}}{-\frac{1}{2}} \\ &= 3\left(\frac{3}{2}\right)^{k} - 2\end{aligned}$$

Sommation 7

$$\begin{aligned}\sum_{i=0}^{\log_3 n} \left(\frac{2}{3}\right)^{i} &= \frac{1-(\frac{2}{3})^{\log_3 n+1}}{1-\frac{2}{3}} \\ &= \frac{1-(\frac{2}{3})^{\log_3 n+1}}{\frac{1}{3}} \\ &= 3 - 2\left(\frac{2}{3}\right)^{\log_3 n}\end{aligned}$$

Sommation 8

$$\begin{aligned}\sum_{i=0}^{\log_3 n} n\left(\frac{2}{3}\right)^{i} &= n(\frac{1-(\frac{2}{3})^{\log_3 n+1}}{1-\frac{2}{3}}) \\ &= n(\frac{1-(\frac{2}{3})^{\log_3 n+1}}{\frac{1}{3}}) \\ &= 3n - 2n\left(\frac{2}{3}\right)^{\log_3 n} \\ &= 3n - 2n(n^{\log_3 \frac{2}{3}}) \\ &= 3n - 2n^{\log_3 2}\end{aligned}$$

Sommation 9

$$\begin{aligned}\sum_{k=2}^{p} k &= \sum_{k=0}^{p} k - \sum_{k=0}^{1} k \\ &= \frac{p(p+1)}{2} - 1 \\ &= \frac{p^2+p-2}{2} \\ &= \frac{(p+2)(p-1)}{2}\end{aligned}$$

Sommation 10

$$\begin{aligned}\sum_{p=0}^{n}(\sum_{p=0}^{n} 5 + \sum_{p=0}^{n} 2) &= \sum_{p=0}^{n}(5(n+1)+2(n+1)) \\ &= \sum_{p=0}^{n}(7n+7) \\ &= \sum_{p=0}^{n} 7n + \sum_{p=0}^{n} 7 \\ &= 7n(n+1) + 7(n+1) \\ &= (n+1)(7n+7) \\ &= 7(n+1)^2\end{aligned}$$

Sommation 11

$$\begin{aligned}\sum_{\theta=0}^{n}(\theta - \sum_{\psi=0}^{m} 2\psi) &= \sum_{\theta=0}^{n}(\theta - m(m+1)) \\ &= \sum_{\theta=0}^{n}\theta - \sum_{\theta=0}^{n}(m(m+1)) \\ &= \frac{n(n+1)}{2} - (m^2n + m^2 + mn + m)) \\ &= \frac{n^2+n-2m^2n-2m^2-2mn-2m}{2}\end{aligned}$$

Sommation 12

$$\begin{aligned}
\sum_{i=1}^{\log n} \log(\frac{n}{2^i}) &= \sum_{i=1}^{\log n} \log n - \sum_{i=1}^{\log n} \log 2^i \\
&= \eta \sum_{i=1}^{\eta} 1 - \sum_{i=1}^{\eta} i && \text{Posons } \eta = \log n \\
&= \eta^2 - \frac{\eta(\eta+1)}{2} \\
&= \frac{\eta^2 - \eta}{2} \\
&= \frac{\log^2 n - \log n}{2}
\end{aligned}$$

Sommation 13

$$\begin{aligned}
\sum_{s=0}^{n-1} s &= \sum_{s=0}^{n} s - n \\
&= \frac{n(n+1)}{2} - n \\
&= \frac{n^2 - n}{2} \\
&= \frac{n(n-1)}{2}
\end{aligned}$$

Sommation 14

$$\begin{aligned}
\sum_{k=2\sigma}^{m} k &= \sum_{k=0}^{m} k - \sum_{k=0}^{2\sigma-1} k \\
&= \frac{m(m+1)}{2} - \frac{2\sigma(2\sigma-1)}{2} \\
&= \frac{m^2 + m - 4\sigma^2 + 2\sigma}{2} \\
&= \frac{(m+2\sigma)(m-2\sigma+1)}{2}
\end{aligned}$$